Vapor Stone
RAIL SYSTEMS
A Wabtec company

10 655 Henri-Bourassa Blvd., West
Saint-Laurent (Québec) Canada
H4S 1A1

Tel: 514.335.4200
Fax: 514.335.4231

www.wabtec.com

Vapor Stone Rail Systems

Generic Software Documentation Project

Project # 7260

Door Control Module application

Software Requirements Specifications (SRS)

Contract Number		CDRL Number	
		Customer Ref. Number	

NOTICE
Plans, specifications and engineering data submitted herewith or which may be submitted, remain the property of **Vapor Stone Rail Systems** and shall not be reproduced or altered or used in whole or in part except by express consent of **VSRS**.

AVIS
Les plans, spécifications et données techniques soumis ci-joints, ou qui peuvent être soumis, restent la propriété de **Vapor Stone Rail Systems** et ne doivent pas être reproduits, altérés ou utilisés, totalement ou partiellement, sans le consentement express de **VSRS**.

	Name / Title	Signature / Date
Prepared :	Thomas Marmeth Software developer	2011-09-19
Verified :	Martin Ouellet, for Eric Malo, Eng. System engineer	2011-09-19
Approved :	Martin Ouellet, Eng. Software department supervisor	2011-09-19

Document No.	**201111270**	Revision	**01**
		Date (yyyy-mm-dd)	**2011-09-19**

Template 200610247-00R-A

Revision List

Rev.	Date	By	ECD	Description of changes
01	2011-09-19	T.Marmeth	---	Released version
0.3	2011-09-19	T.Marmeth	---	Implementing comments from Inspection I-1805 iteration 2
0.2	2011-09-13	T.Marmeth	---	Implementing comments from Inspection I-1805
0.1	2011-09-02	T.Marmeth	---	Implementing comments from Inspection I-1757
0	2011-08-25	T.Marmeth	---	Original issue

Preface

This Software Requirements Specifications (SRS) document complies with the requirements of the IEEE Std. 830-1998 and IEEE Std. 1588-2004.

Table of contents

Revision List 2
Preface....3
Table of contents 4
List of figures 6
List of tables....6

1 INTRODUCTION 7

1.1 Purpose 7
1.2 Scope....7
1.3 Definitions, acronyms, and abbreviations....7
1.3.1 Definitions 7
1.3.2 Acronyms 9
1.4 References 10
1.5 Overview....10
1.6 Maintenance 11
1.7 Configuration Overview 11

2 OVERALL DESCRIPTION 12

2.1 Product Perspective 12
2.1.1 System Interfaces 12
2.1.2 User Interfaces....16
2.1.3 Hardware Interfaces....17
2.1.4 Software Interfaces....22
2.1.5 Communications Interfaces 23
2.1.6 Memory constraints....24
2.1.7 User operations....24
2.1.8 Site adaptation requirements....25
2.2 Product functions....25
2.3 User characteristics 25
2.4 Constraints....25
2.5 Assumptions and dependencies....25
2.6 Apportioning of requirements 26

3 SPECIFIC REQUIREMENTS 27

3.1 External interfaces....27
3.1.1 Inputs & Outputs 29
3.2 Functions 49
3.2.1 Inputs/Outputs management functions 49
3.2.2 Door operator movements functions....51
3.2.3 Emergency Handle Switch Function....55
3.2.4 Door Cutout Function....55
3.2.5 Manage Event log 56
3.2.6 Manage Snapshot Files 57
3.2.7 PTA communication functions 58
3.2.8 Set time....62
Performance requirements 73
3.2.9 Static numerical requirements 73
3.2.10 Dynamic numerical requirements....73
3.3 Logical database requirements 73
3.3.1 Event log file 73
3.3.2 Snapshot file 73
3.3.3 Operating parameters description file....75
3.3.4 Event description file....76
3.3.5 DCM Configuration File....77

3.3.6 Door movement information file 77
3.3.7 System counters files 78
3.4 Design constraints 79
3.4.1 Standards compliance 79
3.5 Software system attributes 79
3.5.1 Reliability 79
3.5.2 Availability 79
3.5.3 Security 79
3.5.4 Maintainability 79
3.5.5 Portability 79

APPENDIX A: CONFIGURABLE PARAMETERS 80

List of figures

Figure 2-1: System Interface Diagram 13
Figure 2-2: General signal opto-isolation – Hardware Interfaces 17
Figure 2-3: Door Operator Motor Control Signals – Hardware Interfaces 18
Figure 2-4: Passenger Alerting Light Signal – Hardware Interfaces 19
Figure 2-5: Passenger Audible Signal – Hardware Interfaces 20
Figure 2-6: Passenger Alerting Light Signal – Hardware Interfaces 20
Figure 2-7: DCL – Hardware Interfaces 22
Figure 3-1: External interface diagram 27
Figure 3-2: Software product perspective 37

List of tables

Table 3-1: Description of the Door Control Trainlines of the Software System 29
Table 3-2: Description of the DO External Interface of the Software System 30
Table 3-3: Description of the DO PSS Inputs of the Software System 31
Table 3-4: Description of the Solenoid Lock I/O of the Software System 32
Table 3-5: Description of the DO LSS Inputs of the Software System 32
Table 3-6: Description of the DO EHS Inputs of the Software System 33
Table 3-7: Description of the No-Motion External Interface of the Software System 33
Table 3-8: Description of the Outside Crew Switch External Interface of the Software System... 33
Table 3-9: Description of the Inside Crew Switch External Interface of the Software System..... 34
Table 3-10: Description of the DPCS External Interface of the Software System 34
Table 3-11: Description of the PAL External Interface of the Software System 35
Table 3-12: Description of the PAS External Interface of the Software System 35
Table 3-13: Description of the SBIL External Interface of the Software System 36
Table 3-14: Description of the Door ID External Interface of the Software System 36
Table 3-15: Mnemonic of HW_COM elements 37
Table 3-16: Description of the Spare Output External Interface of the Software System 41
Table 3-17: Description of the Spare Input External Interface of the Software System 41
Table 3-18: Description of the DEVICEID External Interface of the Software System 42
Table 3-19: Description of the Voltage Monitoring External Interface of the Software System 42
Table 3-20: Description of the DCM Board Temperature External Interface of the Software System 43
Table 3-21: Description of the DCL External Interface of the Software System 43
Table 3-22: Description of the DCM LED External Interface of the Software System 45
Table 3-23: Description of the 7 Segment LED External Interface of the Software System 45
Table 3-24: Mnemonic of memory elements 46
Table 3-25: Snapshot Record Structure 74
Table 3-26: Snapshot Record Header 75
Table 3-27: Operating Parameters Description File Structure 75
Table 3-28: Event Description File Structure 76
Table 3-29: Door Movement Information File Structure 77
Table 3-30: System Counters File Structure 78
Table 4-1: DCM Generic Parameters 80

1 INTRODUCTION

This document is the VAP1 Software Requirements Specifications for the Door Control Module (DCM) application developed by Vapor Stone Rail Systems.

The Door Control Unit is built around two different applications: the Door Control Module application and the Communication Gateway Module application.

Theses 2 applications may run on the same processor as different processes or may run on two different processors. The choice of single or multiple processors is made on a project per project basis and defined in the Software Applications Configuration Specifications (SACS).

This document lists the specifications required to monitor and control one door motor and report the status and health to the Car Network through the Communication Gateway Module application. The Door Control Module application is also capable of receiving messages (and optionally commands) from the train Car Network through the Communication Gateway Module application. The Door Control Module application is part of the microprocessor based system responsible for monitoring and controlling each door motor.

1.1 Purpose

The purpose of this System Requirements Specifications document is to specify the requirements for implementing the Door Control Module application that is part of the Door Control Module.
The intended audience is the Vapor Stone Rail Systems software development and maintenance team members, transit authority and car builders.

1.2 Scope

The scope of this System Requirements Specifications document covers the requirements for the Door Control Module application to control one door motor, monitor the door panel(s), and to report the status and diagnostic information across the Car Network to the monitoring and diagnostics system.

This document specifies the generic specifications for the Door Control Module application. The specifications of the Communication Gateway Module (CGM) application are defined in both the generic Communication Gateway Module Software Requirements Specifications document # 201111272 and the Software Applications Configuration Specifications (SACS). This System Requirements Specifications is complemented with a project specific document, the Software Applications Configuration Specifications (SACS).

1.3 Definitions, acronyms, and abbreviations

Definitions and acronyms listed below establish meaning in the context of this system requirements specifications document.

1.3.1 Definitions

Boot Loader	Software starting the target application of a door module.
Car Network	Data bus connecting the Door Control Module with the rest of the train (others Door Control Modules in the car and others cars).

Communication Gateway Module	A specific processor based electronic module running a Communication Gateway Module application. It acts as gateway between the Door Control Module application and the Car Network/Portable Test Application.
Configuration File	A file within the DCM containing all the configurable parameters value.
Door Module System	A term that is used for generically identifying a processor based electronic module running any software within any DCU.
Door Control Unit	The final unit that is used to control one or more door panels. This unit is normally made of one or two door control modules running the Door Control Module application and possibly one Communication Gateway Module application.
Door Control Module	A specific processor based electronic module running a Door Control Module application. It is used for controlling the door motor.
HW_COM	Communication interface used for communication between Portable Test Application and a door module application, which can be USB, serial or Ethernet.
Seek home mode	When the position of the door is unknown, the door is driven at the slowest speed and stop without reopening in case of obstruction.

1.3.1.1 Event Level Definitions

Event:	Term used to globally identify an Info, an Exception or a Failure.
Info:	Info is an occurrence of an Informative condition monitored by the system, which is neither an exception nor a failure.
Exception:	An exception is an occurrence of a condition that is outside of the specified parameters and affecting an outside Line Replaceable Unit (LRU).
Failure:	A failure is an occurrence of a condition that is outside of the specified parameters and affecting this LRU. A failure normally requires maintenance or replacement to restore affected unit to its normal operation conditions.

1.3.2 Acronyms

BS	British Standards
CB	Circuit Breaker
CG	Communication Gateway
CGM	Communication Gateway Module
CL	Car line
CPU	Central Process Unit
CS	Crew Switch
DCB	Door Control Box
DCL	Door Control Logic
DCM	Door Control Module
DCP	Door Control Panel
DCU	Door Control Unit
DID	Door Identification
DMI	Door Movement Information
DMS	Door Module System
DO	Door Operator
DP	Door Panel
DPCS	Door Panel Cutout Switch
DSR	Door Summary Relay
EDF	Event Description File
EHS	Emergency Handle Switch
EOF	End Of File
FMP	Event Management Plan
HI	High
HW	Hardware
I/O	Input/Output
ICD	Interface Control Document
ICS	Inside Crew Switch
ID	Identification
IEEE	Institute of Electrical and Electronics Engineers, Inc.
ISO	Isolated
LDTS	Local Diagnostic and Test System
LED	Light-Emitting Diode
LO	Low
LRU	Line Replaceable Unit
LS	Lock Switch
MDC	Master Door Controller
MDS	Monitoring and Diagnostic System
MLS	Mechanical Lock Signal
OCS	Outside Crew Switch
OPDF	Operating Parameters Description File
PAL	Passenger Alert Light
PAS	Passenger Audible Signal
PSS	Panel Sensing Switch
PTA	Portable Test Application
PWM	Pulse Width Modulation
RTDM	Real-Time Data Monitoring
Rx	Receive
SACS	Software Applications Configuration Specifications
SBIL	Side Body Indicator Light
SCI	Software Configuration Item

SFD	System Functional Description
SLR	Signal Light Relay
SPI	Serial Peripheral Interface
SRS	System Requirements Specifications
SSL	Sensing Switch Left
SSR	Sensing Switch Right
TL	Trainline
TS	Technical Specifications
Tx	Transmit
UART	Universal Asynchronous Receiver Transmitter
USB	Universal Serial Bus

1.4 References

Standards

IEEE Std 830-1998	IEEE recommended practice for software requirements specifications, 1998-06-25, IEEE, www.ieee.org
IEEE Std 1558-2004	IEEE recommended practice for software documentation for rail equipment and systems, 2005-03-23, IEEE, www.ieee.org
EN 50128	European Standard, Railway applications - Communication, signaling and processing systems - Software for railway control and protection systems BS EN 50128:2001.

Vapor Stone Rail Systems documents

CMM-01	Software enginnering process, VSRS, Rev. 3, 2001-03-31
CMM-02	Software engineering policy manual, VSRS, Rev. 1, 2008-08-17
S17-01	Software requirements specification template, Rev. 0, 2002-01-15
201111274	Portable Test Application Software Requirements Specification Document, VSRS, Rev 0.1
201111271	Boot Loader Application Software Requirements Specification Document, VSRS, Rev 0.1
201010272	Communication Gateway Application Software Requirements Specification Document, VSRS, Rev 0.1
201111489	Portable Test Interface Protocol, Interface Control Document (ICD), VSRS, Rev 0.1
SACS	Software Applications Configuration Specifications, Project's specific
FMP	Fault Management Plan, Project's specific

1.5 Overview

The SRS document describes all inputs into and outputs from the software system. It describes the functionalities and the relationships of these I/Os. It further identifies the responses to abnormal situations when processing these inputs and outputs. The SRS specifies the performance requirements and constraints imposed on the design of the Door Control Module application:

- Section 1: Introduction provides purpose, scope, documentary references and general overview. It also resumes maintenance activities associated with the SRS.

- Section 2: Overall description describes the factors that affect the product and its requirements.

- Section 3: Specific requirements describes all the inputs/outputs relationships and the functionalities.

1.6 Maintenance

Not applicable.

1.7 Configuration Overview

The configurable parameters in this document, which values are Project's specific and are contained in the Configuration File, are:

User configurable parameters:

- SACS.Door.DefaultOpeningDelay
- SACS.Door.DefaultOpeningTime
- SACS.Door.DefaultClosingDelay
- SACS.Door.DefaultClosingTime
- SACS.Door.CrewSwitch.Outside.ClosingDelay
- SACS.Door.CrewSwitch.Inside.ClosingDelay
- SACS.Door.CrewSwitch.Outside.ClosingTime
- SACS.Door.CrewSwitch.Inside.ClosingTime
- SACS.Door.CrewSwitch.Outside.OpeningDelay
- SACS.Door.CrewSwitch.Inside.OpeningDelay
- SACS.Door.CrewSwitch.Outside.OpeningTime
- SACS.Door.CrewSwitch.Inside.OpeningTime
- SACS.Door.Obstruction.Distance
- SACS.Door.Obstruction.Count
- SACS.Door.Obstruction.Range

System configurable parameters:

- SACS.Door.OpeningTime.Range
- SACS.Door.ClosingTime.Range
- SACS.Door.CrewSwitch.LinkTiming
- SACS.Door.Obstruction.EndBehavior
- SACS.Door.AudibleAlarmActivation
- SACS.Door.VisualAlarmActivation
- SACS.Door.SecondaryObstruction.Position
- SACS.Door.SecondaryObstruction.Activation
- SACS.Door.SecondaryObstruction.Delay
- SACS.Door.EmergencyHandle.ClosePower
- SACS.Door.Snapshot.Period
- SACS.Door.Snapshot.Capacity
- SACS.Door.Snapshot.NbOfFiles

Refer to appendix A for more information on each parameter.

2 Overall description

2.1 Product Perspective

The major purpose of the DCM application is to operate a door and to detect and report abnormal conditions.

The DCM application provides the diagnostic and status information to the Car Network via the Communication Gateway Module (CGM) or to the PTA through a serial or Ethernet interface. This information can then be used to analyze the operational process of the door system.

The Monitoring and Control part of the DCM application has software based control capacity over the operation of the doors that is mitigated with hardware.

Events detected by the DCM application are categorized according to their impact on the operation of the subsystem. The events log is stored in non-volatile memory.

The DCM application is a single entity (single binary file) that can be viewed as a single Software Configuration Item (SCI). The DCM application, when present in the DCM, is automatically launched by the Boot Loader application whenever the Boot Loader is detecting correct conditions to start the application. See document 201111271 for more information for the Boot Loader specification.

2.1.1 System Interfaces

The DCM application is directly interfacing to 13 different sub-systems or external devices:

1. The door control trainlines,
2. The Door Operator (DO) sub-system,
3. The No-Motion trainlines,
4. The Outside Crew Switch (OCS) and Inside Crew Switch (optional) devices,
5. The Door Panel Cutout Switch (DPCS) device,
6. The Passenger Alerting Light (PAL) device (optional),
7. The Passenger Audible Signal (PAS) device (optional),
8. The Side Body Indicator Light (SBIL) device (optional),
9. The Door Identification (Door ID) device,
10. The Portable Test Application (PTA) sub-system,
11. The Train Network (configurable). The train network may run on a different hardware or another thread on the DCM CPU.
12. Spares Inputs and Outputs.
13. Internal signals.

The Figure 2-1 below provides an overview of the DCM application interfaces. Please refer to the project specific block diagram drawing for more detailed information.

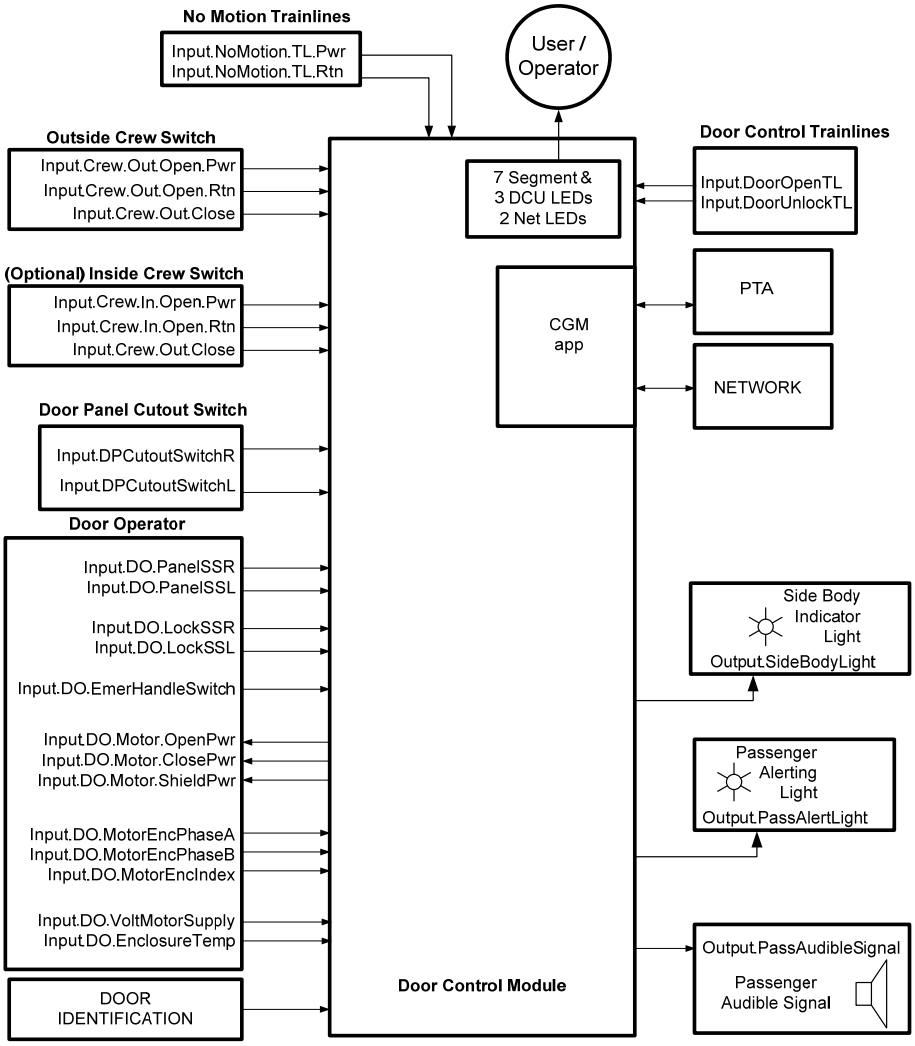

Figure 2-1: System Interface Diagram

2.1.1.1 Door Control Trainline Interface

The Door Control interface is implemented as input signals coming from the trainlines.

The two signals are the Door Open trainline and the Door Unlock trainline that are used together with the No-Motion trainline to command the doors to open and close:

- **Input.DoorOpenTL**: Door Open Trainline is a door open command.
- **Input.DoorUnlockTL**: Door Unlock Trainline is a door unlock command.

<u>Optional:</u>

- **Input.DoorCloseTL**: Door Close Trainline is a door close command.

2.1.1.2 Door Operator (DO) Interface

The Door Operator is a sub-system used for physically moving the door panels. It is implemented as multiple devices that are mounted on a metal panel. The devices of the DO that are used by the DCM are:

- The motor
- The Panel Sensing Switches (PSS)
- The Lock Switches (LS)
- The solenoid lock

The DO interface is implemented as the following signals:

2.1.1.2.1 Motor Related Signals

The motor signals can be separated into four logical groups:

Signals related to the motor power:

- **Input.DO.Motor.OpenPwr:** Open power signal
- **Input.DO.Motor.ClosePwr:** Close power signal
- **Input.DO.Motor.ShieldPwr:** Motor power shield

Signals related to the motor position encoder:

- **Input.DO.MotorEncPhaseA:** Position encoder phase A signal
- **Input.DO.MotorEncPhaseB:** Position encoder phase B signal
- **Input.DO.MotorEncIndex:** Position encoder index signal

Signal related to the motor voltage:

- **Input.DO.VoltMotorSupply:** Motor Voltage Sensor

Signal related to the motor temperature:

- **Input.DO.MotorTemp:** Motor Temperature Sensor

2.1.1.2.2 Panel Sensing Related Signals

These signals are used to detect that the door panels are in their closed position:

- **Input.DO.PanelSSR**: Panel Sensing Switch Right is an input representing the position state of the right door panel.
- **Input.DO.PanelSSL**: Panel Sensing Switch Left is an input representing the position state of the left door panel.

2.1.1.2.3 Solenoid lock Related Signals

- **Input.DO.SolenoidLockPwr**: Solenoid lock power sense.
- **Output.DO.SolenoidDrv**: Output for activating the solenoid used for unlocking the door.

2.1.1.2.4 Lock Sensing Related Signals

These signals are used to detect that the door panels are in their locked state:

- **Input.DO.LockSSR**: Lock Sensing Switch Right is an input representing the state of the right door panel lock switch.
- **Input.DO.LockSSL**: Lock Sensing Switch Left is an input representing the state of the left door panel lock switch.

2.1.1.2.5 Emergency Handle Related Signals

This signal is used to detect an activation of the Emergency Handle.

- **Input.DO.EmerHandleSwitch**: Emergency Handle Switch is an input representing the state of the Emergency Handle.

2.1.1.3 No Motion Trainline Signals

The No-Motion trainlines signals are used by the DCM application to detect that the train speed value is safe to open the doors (normally < 3mph).

For safety reason, the No-Motion trainlines are duplicated:

- **Input.NoMotion.TL.Pwr**: This signal represents the positive part of the No-Motion signal.
- **Input.NoMotion.TL.Rtn**: This signal represents the negative part of the No-Motion signal.

2.1.1.4 Crew Switch Interface

The Crew Switch is a key activated switch that is used to open and close the doors from outside or (optionally) inside the train. For safety reason, the open signal is duplicated (contacts on both positive and negative power signals) while the close signal is not.

<u>Outside Crew Switch:</u>

- **Input.Crew.Out.Open.Pwr**: This signal represents the positive part of the Outside Crew Switch open command.
- **Input.Crew.Out.Open.Rtn**: This signal represents the negative part of the Outside Crew Switch open command.
- **Input.Crew.Out.Close**: This signal is used as a close command from the Outside Crew Switch.

<u>Inside Crew Switch:</u>

Optional:

- **Input.Crew.In.Open.Pwr**: This signal represents the positive part of the Inside Crew Switch open command.
- **Input.Crew.In.Open.Rtn**: This signal represents the negative part of the Inside Crew Switch open command.
- **Input.Crew.In.Close**: This signal is used as a close command from the Inside Crew Switch.

2.1.1.5 Door Panel Cutout Switch (DPCS) Interface

The cutout switches (right and left) are used to cutout a door panel whenever there is a failure of the door system:

- **Input.DPCutoutSwitchR**: This signal represents the cutout status of the right door panel.
- **Input.DPCutoutSwitchL**: This signal represents the cutout status of the left door panel.

2.1.1.6 Passenger Alerting Light (PAL) Interface

The Passenger Alerting Light output signal is used to visually Inform the passenger of the door status.

- **Output.PassAlertLight**: Output used to control the Passenger Alerting Light device.

2.1.1.7 Passenger Audible Signal (PAS) Interface

The Passenger Audible Signal output is used to acoustically Inform the passenger of the door status.

- **Output.PassAudibleSignal**: Output used to control the Passenger Audible Signal device.

2.1.1.8 Side Body Indicator Light (SIBL) Interface

The Side Body Indicator Light is used to inform the train driver that the corresponding car doors is closed and locked.

- **Output.SideBodyLight**: Output used to activate the SIBL.

2.1.1.9 Door Identification (DID) Interface

The door identification inputs are used to uniquely identify a DCU within a car. It is also used to distinguish the type of the car. The door ID connector is external to the DCU allowing replacement of a DCU without reconfiguration.

- **Input.DoorID**: Input used to identify the DCU within a specific car.

2.1.1.10 Motor Type Identification Interface

The Motor Type ID inputs are used to uniquely identify a motor type used by a DCU.

- **Input.MotorTypeID**: Input used to identify the type of motor used for each DCU.

2.1.1.11 PTA Interface

The PTA interface is used to connect a Portable Test Application to the DCU device. It is implemented as an HW_COM port. The configuration and protocol of this interface is partly generic (refer to document 201111489).

2.1.1.12 Network interface

The Network interface, CGM, is used to interface to the car and train network. The configuration and protocol of this interface is project specific.

2.1.2 User Interfaces

The DCM application user interface is composed of one generic interface. Project specific interface can be added.

2.1.2.1 DCM Light Emitting Diodes (LED)

The interface is implemented as an output signal that the user can observe to get the status of the DCM:

- **Output.LED.Run**: The Running LED is a green LED blinking. This shows that the application is operating.
- **Output.LED.Warn**: The Warning LED is a yellow LED that illuminates when at least one event with an exception level is set.
- **Output.LED.Event**: The Event LED is a red LED that illuminates when at least one event with a failure level is set.

2.1.2.2 2 Digit 7 Segment LED (OPTIONAL)

This interface is optional and may not be used in each project.

- **Output.LED.7SEG**: Two digits red segment LED that sequentially display the active Event codes.

2.1.3 Hardware Interfaces

This section is used to describe how the DCM application is accessing the previously defined system interfaces. Please refer to the project specific Car Schematic Block Diagram drawing for more information.

Some signals are opto-isolated. The DCM application is reading the opto-isolated version of the external interface signal. The Figure 2-2 provides a general description of the signals and their opto-isolated counterparts.

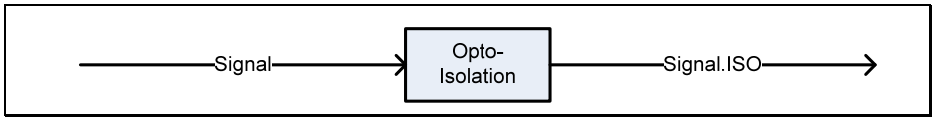

Figure 2-2: General signal opto-isolation – Hardware Interfaces

2.1.3.1 Door Control Trainlines

2.1.3.1.1 Door command trainlines signals

Door open and door unlock signals and their isolated versions.

- **Input.DoorOpenTL:** Isolated → **Input.DoorOpenTL.ISO**
- **Input.DoorUnlockTL:** Isolated → **Input.DoorUnlockTL.ISO**

Optional:

- **Input.DoorCloseTL:** Isolated → **Input.DoorCloseTL.ISO**

2.1.3.2 Door Operator (DO) Sub-System

Motor Signals:

The Figure 2-3 provides a description of the signals used to control and monitor the door motor.

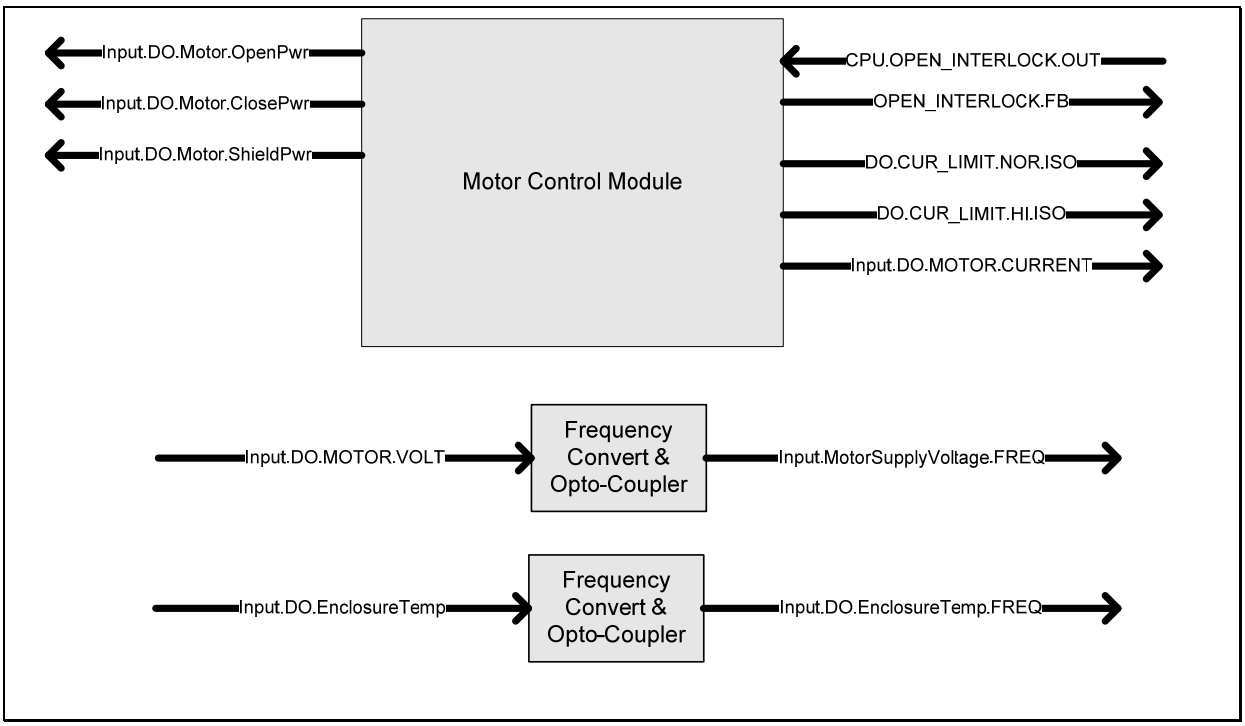

Figure 2-3: Door Operator Motor Control Signals – Hardware Interfaces

- **Input.DO.MotorEncPhaseA:** Isolated → **Input.DO.MotorEncPhaseA.ISO**
- **Input.DO.MotorEncPhaseB:** Isolated → **Input.DO.MotorEncPhaseB.ISO**
- **Input.DO.MotorEncIndex:** Isolated → **Input.DO.MotorEncIndex.ISO**

Panel sensing signals:

Panel sensing switches right and left and their isolated versions.

- **Input.DO.PanelSSR:** Isolated → **Input.DO.PanelSSR.ISO**
- **Input.DO.PanelSSL:** Isolated → **Input.DO.PanelSSL.ISO**

Solenoid lock Related Signals

- **Input.DO.SolenoidLockPwr**: Isolated → **Input.DO.SolenoidLockPwr.ISO**
- **Output.DO.SolenoidDrv**: Isolated → **Output.DO.SolenoidDrv.ISO**

Solenoid lock switches signals:

Right and left solenoid lock switches signals and their isolated versions.

- **Input.DO.LockSSR:** Isolated → **Input.DO.LockSSR.ISO**
- **Input.DO.LockSSL:** Isolated → **Input.DO.LockSSL.ISO**

Emergency handle switch signals:

Emergency handle switch signal and its isolated version:

- **Input.DO.EmerHandleSwitch:** Isolated → **Input.DO.EmerHandleSwitch.ISO**

2.1.3.3 No-Motion Trainlines sub-system

Positive and negative No-Motion trainlines signals and their isolated versions:

- **Input.NoMotion.TL.Pwr:** Isolated → **Input.NoMotion.TL.Pwr.ISO**
- **Input.NoMotion.TL.Rtn**: Isolated → **Input.NoMotion.TL.Rtn.ISO**

2.1.3.4 Crew switch device

Signals used to control the opening and closing of the door from the outside and inside crew switches and their isolated versions:

Exterior Crew switch:

- **Input.Crew.Out.Open.Pwr:** Isolated → **Input.Crew.Out.Open.Pwr.ISO**
- **Input.Crew.Out.Open.Rtn:** Isolated → **Input.Crew.Out.Open.Rtn.ISO**
- **Input.Crew.Out.Close:** Isolated → **Input.Crew.Out.Close.ISO**

Interior Crew switch (Optional):

This interface is optional and may not be used in each project.

- **Input.Crew.In.Open.Pwr:** Isolated → **Input.Crew.In.Open.Pwr.ISO**
- **Input.Crew.In.Open.Rtn:** Isolated → **Input.Crew.In.Open.Rtn.ISO**
- **Input.Crew.In.Close:** Isolated → **Input.Crew.In.Close.ISO**

2.1.3.5 Door Panel Cutout Switch (DPCS) device

Cut-out switches signals and their isolated versions:

- **Input.DPCutoutSwitchR:** Isolated → **Input.DPCutoutSwitchR.ISO**
- **Input.DPCutoutSwitchL:** Isolated → **Input.DPCutoutSwitchL.ISO**

2.1.3.6 Passenger Alerting Light (PAL) device

The Figure 2-4 provides a description of the isolated version of the Passenger Alerting Light control and feedback signals.

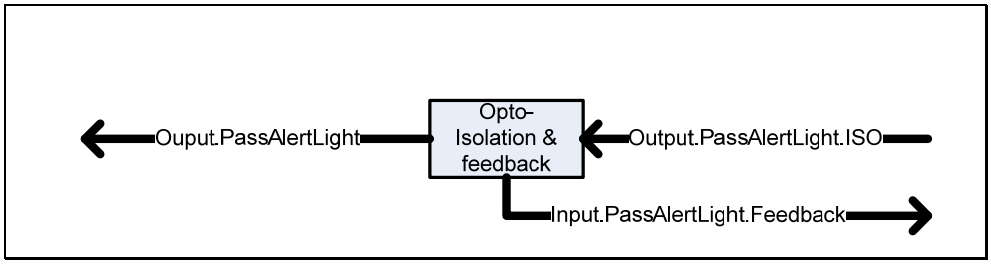

Figure 2-4: Passenger Alerting Light Signal – Hardware Interfaces

2.1.3.7 Passenger Audible Signal (PAS) hardware interfaces

The Figure 2-5 provides a description of the isolated version of the Passenger Audible Signal control and feedback signals.

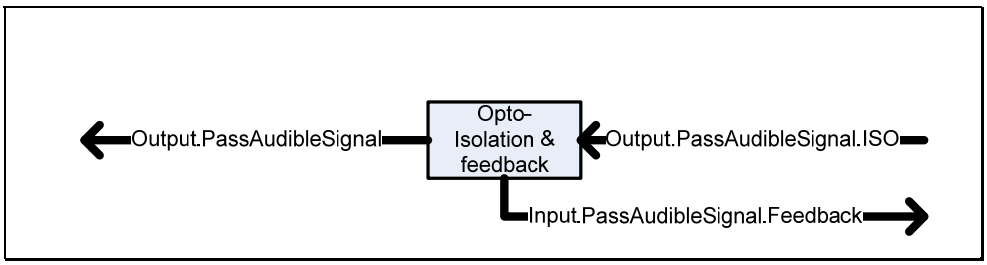

Figure 2-5: Passenger Audible Signal – Hardware Interfaces

2.1.3.8 Side Body Indicator Light (SBIL) device

The Figure 2-6 provides a description of the isolated version of the Side Body Indicator Light control and feedback signals.

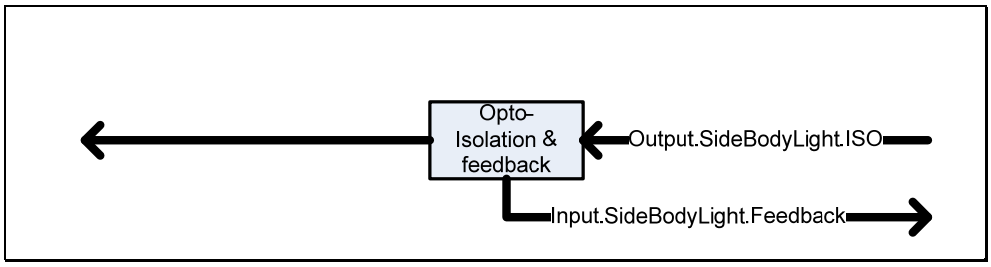

Figure 2-6: Passenger Alerting Light Signal – Hardware Interfaces

2.1.3.9 Door Identification (Door ID) Hardware Interfaces

- **Input.DoorID:** Isolated → **Input.DoorID.ISO**: Input to CPU that differentiate electrical configuration and identifies the DCM number at boot-up time.

2.1.3.10 Motor Identification (Motor ID) Hardware Interfaces

- **Input.MotorTypeID:** Isolated → **Input.MotorTypeID.ISO**: Input to CPU which identifies the motor type used by a DCU.

2.1.3.11 Portable Test Unit (PTU) Hardware Interfaces

The PTU interface HW_COM is implemented as a standard RS-232 line driver using the following signals:

- Rx
- Tx

This interface can be used directly or optionally by a connection through the CGM.

2.1.3.12 Spare Inputs and Outputs

2.1.3.12.1 Spare Outputs Hardware Interface (Optional)

This interface is optional and may not be used in each project.

- **Output.Spare**: Spare output signal connected to the CPU for the future use.
- **Input.Spare.FB**: Spare output feedback signal connected to the CPU for the future use.

2.1.3.12.2 Spare Inputs Hardware Interface (Optional)

This interface is optional and may not be used in each project.

- **Input.Spare1.ISO**: Opto-isolated input signal representing the state of the Spare1 Input
- **Input.Spare2.ISO**: Opto-isolated input signal representing the state of the Spare 2 Input

2.1.3.13 Internal Signals

2.1.3.13.1 Voltage Monitoring

The Voltage Monitoring hardware interface is implemented to verify the valid range of the voltage:

- **Input.VoltMonitLogicSupply**: Input signal representing a valid 5V Logic Supply range to the CPU.
- **Input.VoltMonitCommSupply**: Input signal representing a valid 3.3V Logic Supply range to the CPU.
- **Input.VoltMonitEncSupply**: Input signal representing a valid 5V Encoder Supply range to the CPU.
- **Input.VoltMonitMotorSupply**: Input signal representing a valid 15V Motor Supply range to the CPU.

<u>Optional:</u>

- **Input.VOLTMONIT.VGOPEN**: Input signal representing a valid voltage supply range to the H-Bridge Open.
- **Input.VOLTMONIT.VGCLOSE**: Input signal representing a valid voltage supply range to the H-Bridge Close.

2.1.3.13.2 DCM Board Temperature

The DCM Board Temperature interface is implemented to verify the DCM board temperature range:

- **Input.EnclosureTemp**: Input signal representing a valid DCM temperature range to the CPU.

2.1.3.13.3 Display Hardware Interface (Optional)

This interface is optional and may not be used in each project.

- **Output.LED.7SEG**: Serial display is connected to the CPU and directly shifts the data.

2.1.3.13.4 Memory interface

All memories (Volatile memory, Non-volatile memory and Application memory) interfaces with the CPU through the address and data lines:

- MemoryNonVolatile : Internal or/and external non-volatile memory.
- MemoryVolatile : Internal or/and external volatile memory.
- MemoryApplication: Internal or/and external memory containing the DCM application and the Boot Loader application.

2.1.3.13.5 Door control hardware interface

Door Control Logic (DCL) as shown in the Figure 2-7 performs the following tasks:

- Motor drive logic validation

- Door control interlock logic validation
- Over current shutdown logic validation

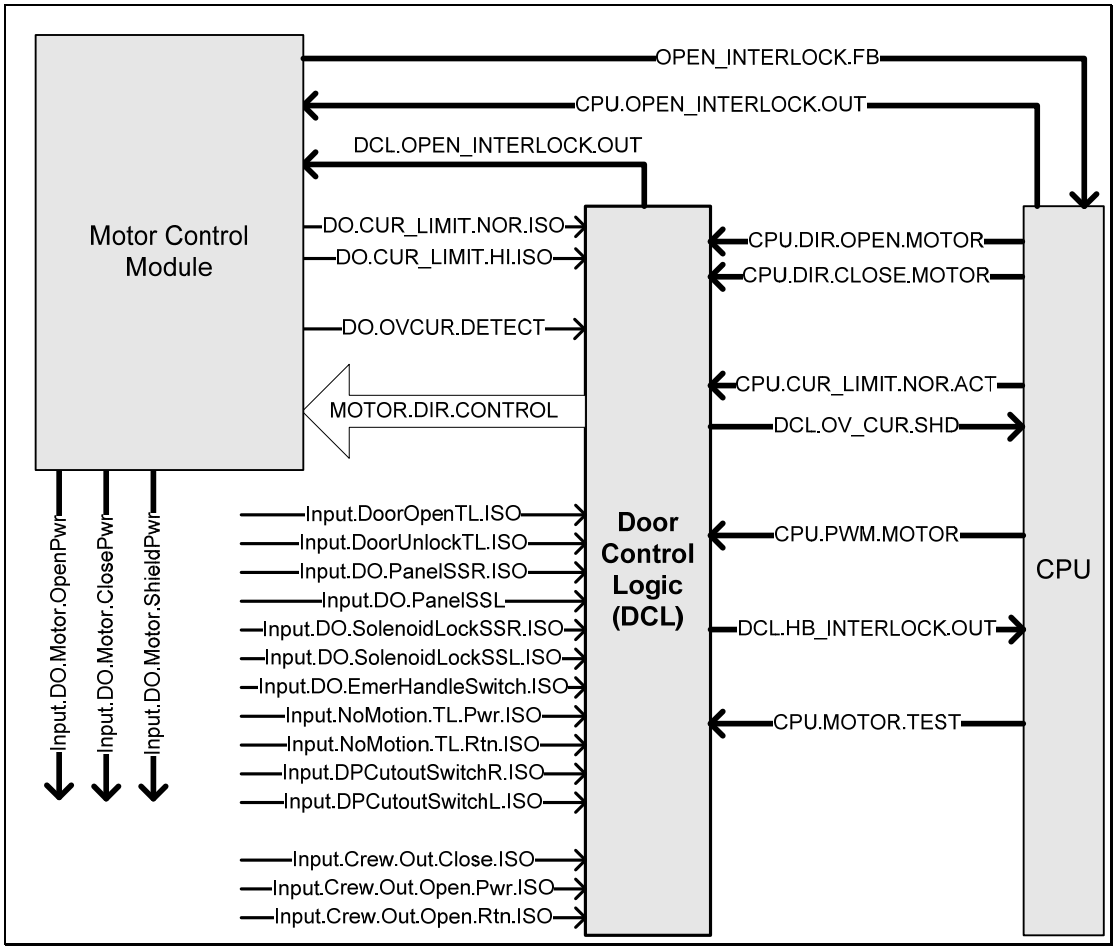

Figure 2-7: DCL – Hardware Interfaces

2.1.4 Software Interfaces

The DCM application accesses:

1) The “Application information” shared memory:
   - Application information shared memory to retrieve information about current version of installed applications.
2) Database interface:
   - Event Log File, see Section 3.4.1 for more detail.
   - Event Description File, see Section 3.4.4 for more detail.
   - Operating Parameter Description File, see 3.4.3 for more detail.
   - Snapshot Files, see Section 3.4.2 for more detail.
   - System Counter File, see Section 3.4.9 for more detail.

The DCM application accesses the DCM Boot Loader application information database to retrieve its size and unique identifier.

2.1.5 Communications Interfaces

2.1.5.1 PTA interface

The DCM application communicates with the PTA (when connected) using HW_COM. The commands applicable to this interface are:

- HW_COM.PTA.Message
- HW_COM.DMS.Message
- HW_COM.PTA.LoginRequest
- HW_COM.DMS.LoginResponse
- HW_COM.PTA.LogoutRequest
- HW_COM.DMS.LogoutResponse
- HW_COM.PTA.RebootRequest
- HW_COM.PTA.HeartbeatRequest
- HW_COM.DCM.HeartbeatResponse
- HW_COM.PTA.SelfTestRequest
- HW_COM.DMS.SelfTestResponse
- HW_COM.PTA.CodeStatusRequest
- HW_COM.DMS.CodeStatusResponse
- HW_COM.PTA.SoftwareVersionsRequest
- HW_COM.DMS.SoftwareVersionsResponse
- HW_COM.PTA.LocalTimeRequest
- HW_COM.DMS.LocalTimeResponse
- HW_COM.PTA.SetTimeRequest
- HW_COM.DMS.SetTimeResponse
- HW_COM.PTA.DoorCommandRequest
- HW_COM.DMS.DoorCommandResponse
- HW_COM.PTA.MonitoringRequest
- HW_COM.DMS.MonitoringResponse
- HW_COM.PTA.EraseEventLogFileRequest
- HW_COM.DMS.EraseEventLogFileResponse
- HW_COM.PTA.EraseSnapshotFileRequest
- HW_COM.DMS.EraseSnapshotFileResponse
- HW_COM.PTA.DeleteCGMLogRequest
- HW_COM.DMS.DeleteCGMLogResponse
- HW_COM.PTA.EventLogFileRequest
- HW_COM.DMS.EventLogFileResponse
- HW_COM.PTA.EventStatusRequest
- HW_COM.DMS.EventStatusResponse
- HW_COM.PTA.DoorMovementinformation FilesRequest
- HW_COM.DMS.DoorMovementinformation FilesResponse
- HW_COM.PTA.SnapshotFileRequest
- HW_COM.DMS.SnapshotFileResponse
- HW_COM.PTA.EventDescriptionFileRequest
- HW_COM.DMS.EventDescriptionFileResponse
- HW_COM.PTA.OperatingParametersDescriptionFileRequest
- HW_COM.DMS.OperatingParametersDescriptionFileResponse
- HW_COM.PTA.ReadDMSConfigurationRequest
- HW_COM.DMS.ReadDMSConfigurationResponse
- HW_COM.PTA.WriteDMSConfigurationRequest
- HW_COM.DMS.WriteDMSConfigurationResponse
- HW_COM.PTA.LocationIdentificationRequest

- HW_COM.DMS.LocationIdentificationResponse
- HW_COM.PTA.CarNumberRequest
- HW_COM.DMS.CarNumberResponse
- HW_COM.PTA.MemoryDumpRequest
- HW_COM.DMS.MemoryDumpResponse
- HW_COM.PTA.SystemCountersRequest
- HW_COM.DMS.SystemCountersResponse
- HW_COM.PTA.SystemCountersResetRequest
- HW_COM.DMS.SystemCountersResetResponse

2.1.5.1.1 Interface Perspective

The communication between the DCM application and the PTA is implemented in a Master – Slave relationship. The DCM application is configured as a Slave. The protocol is based on Request / Response mode. The DCM application shall not send any message to the master end unless it received a request.

2.1.5.1.2 Interface Functions

The following is a summary of the functions available on this interface:

- Login/Logout,
- Event Log Retrieval,
- Snapshot Retrieval,
- Counters Retrieval,
- Real-time monitoring,
- Self-tests result retrieval and 7 segment test,
- Time and date configuration and retrieval,
- Device identification retrieval,
- Applications information retrieval.
- Intercommunication with CGM.

2.1.6 Memory constraints

The memory needs of the DCM application shall not be more than 70 percent of the total installed memory of each type.

2.1.7 User operations

There are no direct user operations done on the DCM application. The user will communicate with the DCM through the PTA or via the Car Network. The user (train operator) will initiate door operations from the DCP, the Emergency Handle or the Crew Key Switch. A local Open Button may be optionally present in the car.

2.1.7.1 Operating Modes

Once initialized, the system is supporting 3 operating modes:

- **MODE.NORMAL:** This is the normal operating mode.
- **MODE.DEGRADED:** This is the degraded operating mode. In this mode, the system may be still able to process and execute the door command depending on the flag conditions. The available flag conditions, each corresponding to a operation mode are:
  - MemoryVolatile.MODE.DEGRADED.DisableOpen

- o MemoryVolatile.MODE.DEGRADED.DisableClose
- o MemoryVolatile.MODE.DEGRADED.ReducedSpeed

Those operations modes are defined in a project per project basis by the SACS or the FMP.

- **MODE.CUTOUT:** This mode is activated whenever the system is detecting that the cut-out switch is activated. In this mode, the system is not processing the door commands.

2.1.8 Site adaptation requirements

No applicable.

2.2 Product functions

The following is a summary of the major functions that the software performs:

- **Initialization function:** Put system in a known good configuration at every boot-up.
- **Hardware polling function:** Read and decode input signals periodically.
- **Primary door control function:** Open, close, and detect obstructions.
- **Status processing function:** Synchronize system status with CGM and PTA.
- **Event detection function:** Detect any unusual system behavior.
- **Event storage function:** Store system events in non-volatile memory to be retrieved by PTA or CGM.
- **SPI communication function:** Exchange valuable system information with CGM processor.
- **PTA interface function**: Send system status; send log, snapshot, event description, and operating parameters files. Receive system configuration parameters.

2.3 User characteristics

The intended users of the DCU should have a good understanding of how the system works, what are the modes of operation, and how to read the status of the DCU. For maintenance, the users should have technical expertise in electronics.

2.4 Constraints

The following lists the factors that limit the software developer's options:

- Software development shall conform to the software engineering process CMMI-01.
- Software development shall conform to the software engineering policy manual CMMI-02.
- Safety issues (keeping the doors closed during motion of the train and when not commanded).

2.5 Assumptions and dependencies

The following lists the factors that can affect the requirements stated in this SRS:

- Change of PTA protocol defined in document #201111489.

2.6 Apportioning of requirements

All generic requirements are addressed in this document complemented with the SACS and FMP. Additional project's specific requirements may also be available in the SACS.

3 Specific requirements

3.1 External interfaces

The Figure 3-1 shows the external interfaces of the DCM application.

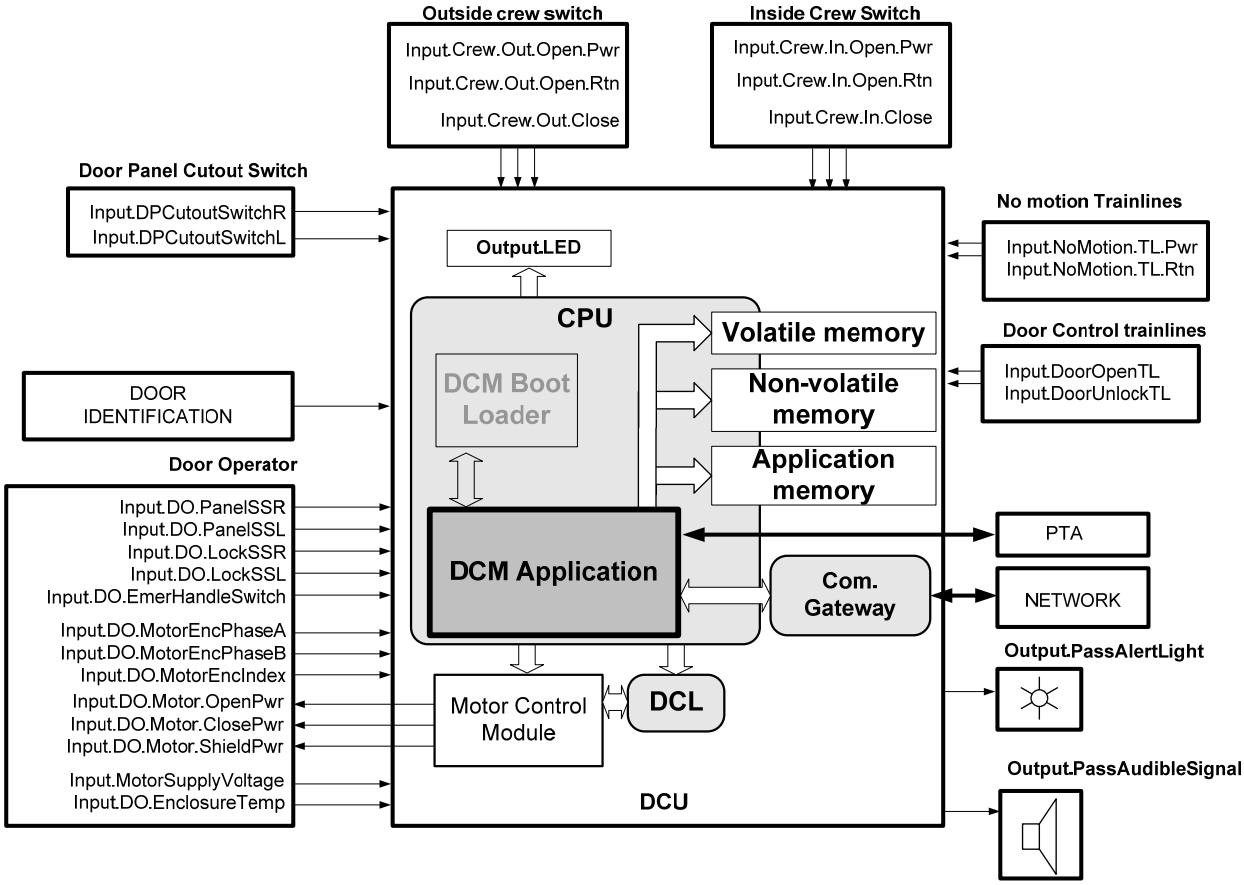

Figure 3-1: External interface diagram

This page has been intentionally left blank

3.1.1 Inputs & Outputs

The state of the inputs can be read by the DCM at different frequencies. Inputs can be digital, analogic or PWM.

3.1.1.1 Door Control Trainlines External Interface

Table 3-1: Description of the Door Control Trainlines of the Software System

Name	Description	Source	Input/Output	Valid Range
Input.DoorOpenTL.ISO	This signal is active when the Open Train Wire is energized.	TL	Input	0: No CMD 1: Open CMD 1→0: Close CMD
Input.DoorUnlockTL.ISO	This signal is active when the Unlock Train Wire is energized.	TL	Input	0: No CMD 1: Open CMD 1→0: Close CMD
Input.DoorCloseTL.ISO	OPTIONAL This signal is active when the Close Train Wire is energized.	TL	Input	0: No CMD 1: Open CMD 1→0: Close CMD

3.1.1.2 Door Operator (DO) External Interface

3.1.1.2.1 Motor Signals

Table 3-2: Description of the DO External Interface of the Software System

Name	Description	Source	Input/Output	Valid Range
CPU.OPEN_INTERLOCK.OUT	This signal is active when door opening condition is validated.	CPU	Output	0: Inactive 1: Active
OPEN_INTERLOCK.FB	This feedback signal is active when both DCL.OPEN_ INTERLOCK.OUT and CPU.OPEN_INTERLOCK.OUT are active.	Internal	Input	0: Inactive 1: Active
DO.CUR_LIMIT.NOR.ISO	This input is a pulse for the duration of the over current, when the motor current is higher than the normal limit.	DO	Input	0: Inactive 1: Active
DO.CUR_LIMIT.HI.ISO	This input is a pulse for the duration of the over current, when the motor current is higher than the high limit.	DO	Input	0: Inactive 1: Active
Input.DO.MOTOR.CURRENT	Motor current (signed) read from the motor drive. This signal is an analog value that represents the current of the motor.	DO	Input	100mV/Amp, 2.5V=>0A 4.5V=>20A
Input.DO.MOTOR.VOLT	Motor Voltage. This signal is an analog value that represents the voltage of the motor.	DO	Input	-
Input.DO.MotorTemp	Motor Temperature. This signal is an analog value that represents the temperature of the motor.	DO	Input	- -
Input.MotorSupplyVoltage.FREQ	Signal proportional to the filtered Motor Power.	DO	Input	41.63µs=>20V 15.93µs=>50V

Name	Description	Source	Input/Output	Valid Range
Input.DO.EnclosureTemp.FREQ	Frequency representing the Door Operator environment temperature. This signal is also inactive when the motor power is unavailable in the door operator and in the DCM input circuit..	DO	Input	2096Hz=>-45°C 6452Hz=>150°C
Input.DO.MotorEncPhaseA.ISO	Motor Position Monitoring to indicate the position of the door. This signal is also inactive when the motor power is unavailable in the door operator and in the DCM input circuit.	DO	Input	0: Inactive 1: Active
Input.DO.MotorEncPhaseB.ISO	Motor Position Monitoring to indicate the position of the door. This signal is also inactive when the motor power is unavailable in the door operator and in the DCM input circuit.	DO	Input	0: Inactive 1: Active
Input.DO.MotorEncIndex.ISO	Motor encoder index pulsed once per motor revolution. This signal is also inactive when the motor power is unavailable in the door operator and in the DCM input circuit.	DO	Input	0: Inactive 1: Active

3.1.1.2.2 Panel Sensing Signals

Table 3-3: Description of the DO PSS Inputs of the Software System

Name	Description	Source	Input/Output	Valid Range
Input.DO.PanelSSL.ISO	This sensor is deactivated when the left door is in close position (almost locked). This signal is also inactive when the motor power is unavailable in the door operator and in the DCM input circuit.	DO	Input	0: Inactive 1: Active
Input.DO.PanelSSR.ISO	This sensor is deactivated when the right door is in close position (almost locked). This signal is also inactive when the motor power is unavailable in the door operator and in the DCM input circuit.	DO	Input	0: Inactive 1: Active

3.1.1.2.3 Solenoid Lock Signals

Table 3-4: Description of the Solenoid Lock I/O of the Software System

Name	Description	Source	Input/Output	Valid Range
Input.DO.SolenoidLockPwr.ISO	This sensor is activated when there is power at the solenoid lock	DO	Input	0: Inactive 1: Active
Output.DO.SolenoidDrv.ISO	This output activates the solenoid lock.	DO	Input	0: Inactive 1: Active

3.1.1.2.4 Lock Sensing Signals

Table 3-5: Description of the DO LSS Inputs of the Software System

Name	Description	Source	Input/Output	Valid Range
Input.DO.LockSSL.ISO	This signal is deactivated when the left door is locked. This signal is also inactive when the motor power is unavailable in the door operator and in the DCM input circuit.	DO	Input	0: Inactive 1: Active
Input.DO.LockSSR.ISO	This signal is deactivated when the right door is locked. This signal is also inactive when the motor power is unavailable in the door operator and in the DCM input circuit.	DO	Input	0: Inactive 1: Active

3.1.1.2.5 Emergency Handle Switch Signals

Table 3-6: Description of the DO EHS Inputs of the Software System

Name	Description	Source	Input/Output	Valid Range
Input.DO.EmerHandleSwitch.ISO	This signal is active when Emergency Handle is pulled. This signal is also inactive when the motor power is unavailable in the door operator and in the DCM input circuit.	DO	Input	0: Inactive 1: Active

3.1.1.3 No-Motion Trainlines External Interface

Table 3-7: Description of the No-Motion External Interface of the Software System

Name	Description	Source	Input/Output	Valid Range
Input.NoMotion.TL.Pwr.ISO	This signal is active when the No-Motion Interlock Trainline Power input is energized.	DO	Input	0: Inactive 1: Active
Input.NoMotion.TL.Rtn.ISO	This signal is active when the No-Motion Interlock Trainline Return input is energized.	DO	Input	0: Inactive 1: Active

3.1.1.4 Outside Crew Switch External Interface

Table 3-8: Description of the Outside Crew Switch External Interface of the Software System.

Name	Description	Source	Input/Output	Valid Range
Input.Crew.Out.Open.Pwr.ISO	This signal is active when the OCS Door Open Power is energized.	OCS	Input	0: Inactive 1: Active

Name	Description	Source	Input/Output	Valid Range
Input.Crew.Out.Open.Rtn.ISO	This signal is active when the OCS Door Open Return is energized.	OCS	Input	0: Inactive 1: Active
Input.Crew.Out.Close.ISO	This signal is active when the OCS Door Close is energized.	OCS	Input	0: Inactive 1: Active

3.1.1.5 [OPTIONAL] Inside Crew Switch External Interface

Table 3-9: Description of the Inside Crew Switch External Interface of the Software System.

Name	Description	Source	Input/Output	Valid Range
Input.Crew.In.Open.Pwr.ISO	OPTIONAL This signal is active when the ICS Door Open Power is energized.	ICS	Input	0: Inactive 1: Active
Input.Crew.In.Open.Rtn.ISO	OPTIONAL This signal is active when the ICS Door Open Return is energized.	ICS	Input	0: Inactive 1: Active
Input.Crew.In.Close.ISO	OPTIONAL This signal is active when the ICS Door Close is energized.	ICS	Input	0: Inactive 1: Active

3.1.1.6 Door Panel Cutout Switch (DPCS) External Interface

Table 3-10: Description of the DPCS External Interface of the Software System

Name	Description	Source	Input/Output	Valid Range
Input.DPCutoutSwitchL.ISO	This signal is active when the Door Panel Cutout Switch Left signal is in	DPCS	Input	0: Inactive

Name	Description	Source	Input/Output	Valid Range
	Cut-out (summary loop bypassed) position.			1: Active
Input.DPCutoutSwitchR.ISO	This signal is active when the Door Panel Cutout Switch Right signal is in Cut-out (summary loop bypassed) position.	DPCS	Input	0: Inactive 1: Active

3.1.1.7 Passenger Alerting Light (PAL) External Interface

Table 3-11: Description of the PAL External Interface of the Software System

Name	Description	Source	Input/Output	Valid Range
Output.PassAlertLight.ISO	This output signal activates Passenger Alerting Light.	PAL	Outputs	0: Inactive 1: Active
Input.PassAlertLight.Feedback	This feedback input is activated when the Output.PassAlertLight.ISO is active.	PAL	Input	0: Inactive 1: Active

3.1.1.8 Passenger Audible Signal (PAS) External Interface

Table 3-12: Description of the PAS External Interface of the Software System

Name	Description	Source	Input/Output	Valid Range
Output.PassAudibleSignal.ISO	This output signal activates Passenger Audible Signal.	PAS	Outputs	0: Inactive 1: Active
Input.PassAudibleSignal.Feedback	This feedback input is activated when the Output.PassAudibleSignal.ISO is active.	PAS	Input	0: Inactive 1: Active

3.1.1.9 Side Body indicator Light (SBIL) External Interface

Table 3-13: Description of the SBIL External Interface of the Software System

Name	Description	Source	Input/Output	Valid Range
Output.SideBodyLight.ISO	This output signal activates Side Body Indicator Light.	SBIL	Outputs	0: Inactive 1: Active
Input. SideBodyLight.Feedback	This feedback input is activated when the Output.SideBodyLight.ISO is active.	SBIL	Input	0: Inactive 1: Active

3.1.1.10 Door Identification (Door ID) External Interface

Table 3-14: Description of the Door ID External Interface of the Software System

Name	Description	Source	Input/Output	Valid Range
Input.DoorID	This input is used to identify the location of the DCU in the car.	DID	Input	DCU1 DCU2 DMU3 DMU4

3.1.1.11 HW_COM

Figure 3-2 depicts the software system logical interfaces and shows how the PTA interfaces with CPU n°01/CPU n°X Boot Loader and DCM application via HW_COM. In the case more than 1 CPU are used for one DCU, a HW_COM port has to be shared between several controllers inside the DCU. A mechanism shall be implemented to avoid conflict between the multiples controllers. The messages sent and received on this interface are listed in Table 3-15.

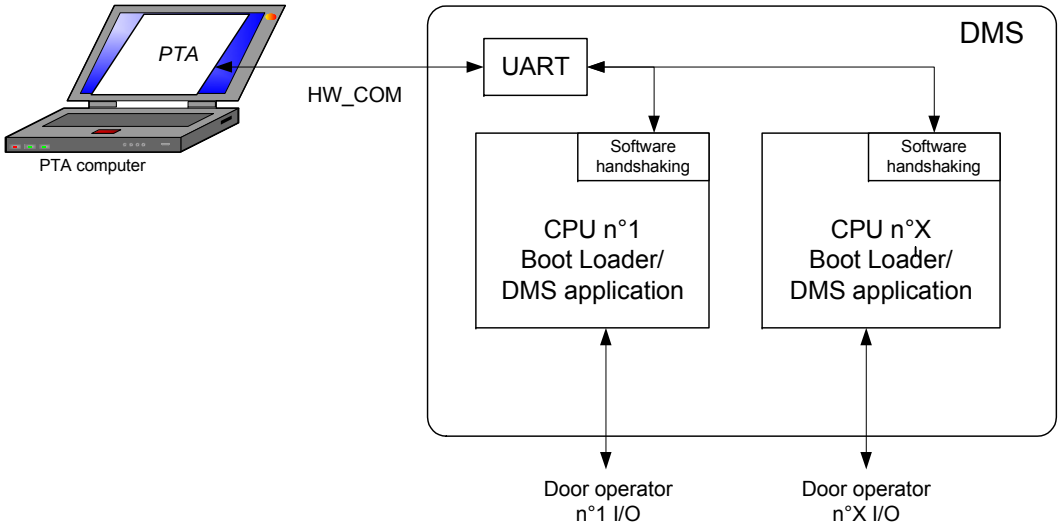

Figure 3-2: Software product perspective

Table 3-15: Mnemonic of HW_COM elements

Name	Description	Input/ Output
HW_COM.PTA.Message	A generic request message sent by the PTA	Input
HW_COM.DMS.Message	A generic request message sent by the DMS	Output
HW_COM.PTA.LoginRequest	A request to log into the DCM sent by the PTA	Input
HW_COM.DMS.LoginResponse	A response of the DCM to HW_COM.PTA.LoginRequest	Output
HW_COM.PTA.LogoutRequest	A request to log out of the DCM sent by the PTA	Input

Name	Description	Input/ Output
HW_COM.DMS.LogoutResponse	A response of the DCM to HW_COM.PTA.LogoutRequest	Output
HW_COM.PTA.RebootRequest	A request to reboot the DCM sent by the PTA	Input
HW_COM.PTA.HeartbeatRequest	A request of the PTA to verify the presence of the PTA.	Input
HW_COM.DMS.HeartbeatResponse	A response of the DCM to HW_COM.DMS.HeartbeatRequest	Output
HW_COM.PTA.SelfTestRequest	A request to self test the DCM sent by the PTA	Input
HW_COM.DMS.SelfTestResponse	A response of the DCM to HW_COM.PTA.SelfTestRequest	Output
HW_COM.PTA.CodeStatusRequest	A request for code status sent by the PTA	Input
HW_COM.DMS.CodeStatusResponse	A response of the DCM to HW_COM.PTA.CodeStatusRequest	Output
HW_COM.PTA.SoftwareVersionsRequest	A request for the software versions sent by the PTA	Input
HW_COM.DMS.SoftwareVersionsResponse	A response of the DCM to HW_COM.PTA.SoftwareVersionsRequest	Output
HW_COM.PTA.LocalTimeRequest	A request for the local DCM time sent by the PTA	Input
HW_COM.DMS.LocalTimeResponse	A response of the DCM to HW_COM.PTA.LocalTimeRequest	Output
HW_COM.PTA.SetTimeRequest	A request to change the DCM time sent by the PTA	Input
HW_COM.DMS.SetTimeResponse	A response of the DCM to HW_COM.PTA.SetTimeRequest	Output
HW_COM.PTA.DoorCommandRequest	A request containing a door command sent by the PTA	Input
HW_COM.DMS.DoorCommandResponse	A response of the DCM to HW_COM.PTA.DoorCommandRequest	Output
HW_COM.PTA.MonitoringRequest	A request to start the real-time monitoring sent by the PTA	Input
HW_COM.DMS.MonitoringResponse	A response of the DCM to HW_COM.PTA.MonitoringRequest	Output

Name	Description	Input/ Output
HW_COM.PTA.DeleteCGMLogRequest	A request to delete the CGMlog sent by the PTA	Input
HW_COM.DMS.DeleteCGMLogResponse	A response of the DCM to HW_COM.PTA.DeleteCGMLogRequest	Output
HW_COM.PTA.EventLogFileRequest	A request for the event log file sent by the PTA	Input
HW_COM.DMS.EventLogFileResponse	A response of the DCM to HW_COM.PTA.EventLogFileRequest	Output
HW_COM.PTA.EraseEventLogFileRequest	A request to erase the event log files sent by the PTA	Input
HW_COM.DMS.EraseEventLogFileResponse	A response of the DCM to HW_COM.PTA.EraseLogFileRequest	Output
HW_COM.PTA.EventStatusRequest	A request for the event status sent by the PTA	Input
HW_COM.DMS.EventStatusResponse	A response of the DCM to HW_COM.PTA.EventStatusRequest	Output
HW_COM.PTA.DoorMovementinformation FilesRequest	A request for the door movement information file sent by the PTA	Input
HW_COM.DMS.DoorMovementinformation FilesResponse	A response of the DCM to HW_COM.PTA.DoorMovementinformation FilesRequest	Output
HW_COM.PTA.SnapshotFileRequest	A request for the snapshot file sent by the PTA	Input
HW_COM.DMS.SnapshotFileResponse	A response of the DCM to HW_COM.PTA.SnapshotFileRequest	Output
HW_COM.PTA.EraseSnapshotFileRequest	A request to erase the snapshot files sent by the PTA	Input
HW_COM.DMS.EraseSnapshotFileResponse	A response of the DCM to HW_COM.PTA.EraseSnapshotFileRequest	Output
HW_COM.PTA.EventDescriptionFileRequest	A request for the event description file sent by the PTA	Input
HW_COM.DMS.EventDescriptionFileResponse	A response of the DCM to HW_COM.PTA.EventDescriptionFileRequest	Output
HW_COM.PTA.OperatingParametersDescriptionFileRequest	A request for the operating parameters description file sent by the PTA	Input

Name	Description	Input/ Output
HW_COM.DMS.OperatingParametersDescriptionFileResponse	A response of the DCM to HW_COM.PTA.OperatingParametersDescriptionFileRequest	Output
HW_COM.PTA.ReadDMSConfigurationRequest	A request for reading the DCM configuration file sent by the PTA	Input
HW_COM.DMS.ReadDMSConfigurationResponse	A response of the DCM to HW_COM.PTA.ReadDMSConfigurationRequest	Output
HW_COM.PTA.WriteDMSConfigurationRequest	A request for writing into the DCM configuration file sent by the PTA	Input
HW_COM.DMS.WriteDMSConfigurationResponse	A response of the DCM to HW_COM.PTA.WriteDMSConfigurationRequest	Output
HW_COM.PTA.LocationIdentificationRequest	A request for the location identification sent by the PTA	Input
HW_COM.DMS.LocationIdentificationResponse	A response of the DCM to HW_COM.DMS.LocationIdentificationResponse	Output
HW_COM.PTA.CarNumberRequest	A request for the car number sent by the PTA	Input
HW_COM.DMS.CarNumberResponse	A response of the DCM to HW_COM.PTA.CarNumberRequest	Output
HW_COM.PTA.MemoryDumpRequest	A request for the MemoryVolatile.Content sent by the PTA	Input
HW_COM.DMS.MemoryDumpResponse	A response of the DCM to HW_COM.DMS.MemoryDumpResponse	Output
HW_COM.PTA.SystemCountersRequest	A request for the system counters sent by the PTA	Input
HW_COM.DMS.SystemCountersResponse	A response of the DCM to HW_COM.PTA.SystemCountersRequest	Output
HW_COM.PTA.SystemCountersResetRequest	A request for resetting system counters sent by the PTA	Input
HW_COM.DMS.SystemCountersResetResponse	A response of the DCM to HW_COM.PTA.SystemCountersResetRequest	Output

3.1.1.12 Spare Output External Interface

Table 3-16: Description of the Spare Output External Interface of the Software System

Name	Description	Source	Input/Output	Valid Range
Output.SPARE	Spare output for the future use.	CPU	Output	0: Inactive 1: Active
Output.SPARE.FB	Spare output feedback for the future use.	SPARE.OUT	Input	0: Inactive 1: Active

3.1.1.13 Spare Input External Interface

Table 3-17: Description of the Spare Input External Interface of the Software System

Name	Description	Source	Input/Output	Valid Range
Input.SPARE1.ISO	This signal is activated when the Spare Input 1 is energized.	SPARE	Input	0: Inactive 1: Active
Input.SPARE2.ISO	This signal is activated when the Spare Input 2 is energized.	SPARE	Input	0: Inactive 1: Active

3.1.1.14 Internal Signals

3.1.1.14.1 Device Identification (DeviceID) External Interface

Table 3-18: Description of the DEVICEID External Interface of the Software System

Name	Description	Source	Input/Output	Valid Range
Input.DeviceID	This input is used to differentiate DCM from No-Motion trainlines.	Internal	Input	0: DCM 1: Trainline

3.1.1.14.2 Voltage Monitoring External Interface

Table 3-19: Description of the Voltage Monitoring External Interface of the Software System

Name	Description	Source	Input/Output	Valid Range
Input.VoltMonitLogicSupply	Indicates the supply voltage of the logic/CPU is within boundaries.	Voltage Monitoring	Input	0: Inactive 1: Active
Input.VoltMonitCommSupply	Indicates the supply voltage of the logic/CPU is within boundaries.	Voltage Monitoring	Input	0: Inactive 1: Active
Input.VoltMonitEncSupply	Indicates the supply voltage of the Encoder circuit is within boundaries.	Voltage Monitoring	Input	0: Inactive 1: Active
Input.VoltMonitMotorSupply	Indicates the supply voltage of the motor drive is within boundaries.	Voltage Monitoring	Input	0: Inactive 1: Active

Name	Description	Source	Input/Output	Valid Range
Input.VOLTMONIT.VGOPEN	Indicates the supply voltage of the H-Bridge Open is within boundaries.	Voltage Monitoring	Input	0: Inactive 1: Active
Input.VOLTMONIT.VGCLOSE	Indicates the supply voltage of the H-Bridge Close is within boundaries.	Voltage Monitoring	Input	0: Inactive 1: Active

3.1.1.14.3 DCM Board Temperature External Interface

Table 3-20: Description of the DCM Board Temperature External Interface of the Software System

Name	Description	Source	Input/Output	Valid Range
Input.DO.EnclosureTemp	Analog voltage input representing the DCM board temperature.	DCM Temperature	Input	1°C/10mV 0.5V=>0°C 2.0V=>150°C

3.1.1.14.4 Door Control Logic External Interface

Table 3-21: Description of the DCL External Interface of the Software System

Name	Description	Source	Input/Output	Valid Range
CPU.DIR.OPEN.MOTOR	This signal is used to drive the door in the open direction.	CPU	Output	0: Inactive 1: Active
CPU.DIR.CLOSE.MOTOR	This signal is used to drive the door in the close direction	CPU	Output	0: Inactive 1: Active

Name	Description	Source	Input/Output	Valid Range
CPU.CUR_LIMIT.NOR.ACT	This signal is used to select the normal current limit.	CPU	Output	0: Inactive 1: Active
DCL.OV_CUR.SHD	This input is active when the motor drive is shutdown after running close to its maximum value for 2 seconds	DCL	Input	0: Inactive 1: Active
CPU.PWM.MOTOR	This is the pulse width modulation signal that is generated by the DCM application to open or close the door.	CPU	Output	24KHz
DCL.HB_INTERLOCK.OUT	This signal is active when DCL confirms the door control logic to the DCM Application.	DCL	Input	0: Inactive 1: Active
CPU.MOTOR.TEST	This signal is active to validate the H-Bridge motor control module before opening the door.	CPU	Output	0: Inactive 1: Active
DCL.DMS.DATA	Serial Data input is used to read the serial signal from DCL.	DCL Serial Link	Input	-
CPU. DCL.CLOCK	Serial Clock sent to the DCL to retrieve the DCL Serial Data.	DCL Serial Link	Output	-
CPU. DCL.SPARE1	Output connected to the DCM DCL for future use.	CPU	Outputs	0: Inactive 1: Active
DCL.CPU.SPARE2	Input connected to the DCM DCL for future use.	DCL	Input	0: Inactive 1: Active

3.1.1.14.5 DCM LED External Interface

Table 3-22: Description of the DCM LED External Interface of the Software System

Name	Description	Source	Input/Output	Valid Range
Output.LED.Run	Green LED flashes when the DCM Application is running.	CPU	Outputs	0: Inactive 1: Active
Output.LED.Warn	Active when an exception occurs in the Event Log causing yellow LED to illuminate.	CPU	Outputs	0: Inactive 1: Active
Output.LED.Event	Active when a failure occurs in the Event Log causing red LED to illuminate.	CPU	Outputs	0: Inactive 1: Active
Output.LED.TEST	Active at boot-up to test all DCM LEDs.	CPU	Outputs	0: Inactive 1: Active

3.1.1.14.6 [OPTIONAL] Two Digit 7 Segment LED External Interface

Table 3-23: Description of the 7 Segment LED External Interface of the Software System

Name	Description	Source	Input/Output	Valid Range
Output.LED.7SEG	OPTIONAL Displaying the active Event number	CPU	Output	0: Inactive 1: Active

3.1.1.14.7 Memory

Each DCM CPU can interface with a maximum of 3 types of memories: volatile memory, non-volatile memory and application memory. It is possible that one or more of these memories will not be used by the DMS.

Table 3-24: Mnemonic of memory elements

Name	Description	Source	Input/Output
MemoryNonVolatile	A non-volatile memory	Non-volatile memory	-
MemoryNonVolatile.Content	All memory cells and address lines of the non-volatile memory	Non-volatile memory	-
MemoryNonVolatile.DCMApplicationIntegrityCheck	Integrity check result of DCM application	Non-volatile memory	-
MemoryNonVolatile.DCMApplicationCodeSize	DCM application software code size	Non-volatile memory	-
MemoryNonVolatile.SCIVersion	DCM and Boot Loader application software version	Non-volatile memory	-
MemoryNonVolatile.BootLoaderApplicationIntegrityCheck	Integrity check result of Boot Loader application	Non-volatile memory	-
MemoryNonVolatile.BootLoaderApplicationCodeSize	Boot Loader software code size	Non-volatile memory	-
MemoryNonVolatile.EventDescriptionFile	Event description file, containing events description and parameters.	Non-volatile memory	-

Name	Description	Source	Input/Output
MemoryNonVolatile.EventDescriptionFileVersions	Event description file version	Non-volatile memory	-
MemoryNonVolatile.OperatingParametersDescriptionFile	Operating parameters description file, containing the parameters values used by the door, listed in Table 2-1	Non-volatile memory	-
MemoryNonVolatile.OperatingParametersDescriptionFileVersion	Operating parameters description file version	Non-volatile memory	-
MemoryNonVolatile.DoorMovementInformationFiles	Door movement information file, used to store information about latest opening and closing sequence.	Non-volatile memory	-
MemoryNonVolatile.EventLogFile	Event log file, containing, information about events detected by the system.	Non-volatile memory	-
MemoryNonVolatile.EventStatus	Event status, containing the status (active or not) of all events into the system.	Non-volatile memory	-
MemoryNonVolatile.SnapshotFile	Snapshot file, containing details about all signals valuable for DCM diagnostic.	Non-volatile memory	-
MemoryNonVolatile.DCMConfigurationFile	DCM configuration file, containing opening time, closing time, and obstruction retries number.	Non-volatile memory	-
MemoryNonVolatile.SystemCountersFile	System counters file, containing counters of the door cycles, the operating hours and the event occurrence	Non-volatile memory	-
MemoryVolatile	A volatile memory	Volatile memory	-
MemoryVolatile.Content	All memory cells and address lines of the volatile memory	Volatile memory	-
MemoryVolatile.SelfTestResults	Temporary SelfTestResults kept in a volatile memory buffer	Volatile memory	-

Name	Description	Source	Input/Output
MemoryVolatile.SnapshotBuffer	Temporary snapshot data kept in a volatile memory buffer	Volatile memory	-
MemoryVolatile.PositionEncoderCount	Position encoder count, used to determine door position	Volatile memory	-
MemoryVolatile.LocalTime	Local time	Volatile memory	-
MemoryVolatile.LoginState	The login state of the PTA connected to the DCM (Connected or Disconnected)	Volatile memory	-
MemoryVolatile.LocationIdentification	Local identification, identifier used to specify door location into a car.	Volatile memory	-
MemoryVolatile.CarNumber	Car number, identifier used to specify car location into the train.	Volatile memory	-
MemoryVolatile.MODE_DEGRADED.DisableOpen	The state of the disable open flag, a degraded mode for the door movement	Volatile memory	-
MemoryVolatile.MODE_DEGRADED. DisableClose	The state of the disable close flag, a degraded mode for the door movement	Volatile memory	-
MemoryVolatile.MODE_DEGRADED. ReduceSpeed	The state of the disable reduce speed flag, a degraded mode for the door movement	Volatile memory	-
MemoryApplication	A memory containing the DCM and Boot Loader applications	Application memory	-
MemoryApplication.DCMApplication	DCM application code	Application memory	-
MemoryApplication.BootLoaderApplication	Boot loader application code	Application memory	-

3.2 Functions

The following sections describe the actions that take place within the software in order to process inputs and outputs.

3.2.1 Inputs/Outputs management functions

3.2.1.1 VAP1.1 Test DCM components

a) Purpose

The self-test is used to verify all components used by DCM application.

b) Data

Source	Input
MemoryVolatile	MemoryVolatile
MemoryVolatile	MemoryNonVolatile
MemoryVolatile	MemoryVolatile.Content
MemoryNonVolatile	MemoryNonVolatile.Content
MemoryApplication	MemoryApplication.DCMApplication
MemoryApplication	MemoryApplication.BootLoaderApplication
CGM	CGM.CommunicationStatus

Destination	Output
Memory	MemoryVolatile.SelfTestResults
Memory	MemoryNonVolatile.EventLogFile

c) Processing

1. VAP1.1.2 On system startup and continuously, at a slow rate (one test per second) and with a minimum of one test per day, DCM application shall execute the self-test routine, testing the MemoryVolatile and MemoryNonVolatile, the DCM and Boot loader application integrities and the CGM communication status.

   - VAP1.1.2.1 MemoryVolatile and MemoryNonVolatile

     – VAP1.1.2.1.1 The self-test shall verify that MemoryVolatile and MemoryNonVolatile can read and write data without generating any error.

     – VAP1.1.2.1.2 The self-test shall verify the integrity of MemoryVolatile.Content and MemoryNonVolatile.Content.

   - VAP1.1.2.2 DCM application integrity check

     VAP1.1.2.2.1 The self-test shall check the integrity of the MemoryApplication.DCMApplication.

   - VAP1.1.2.3 Boot Loader integrity check

     VAP1.1.2.3.1 The self-test shall check the integrity of the MemoryApplication.BootLoaderApplication.

   - VAP1.1.2.5 CGM communication status

- VAP1.1.2.5.1 The self-test shall verify CGM.CommunicationStatus with the CGM application.

2. VAP1.1.3 The result of self-tests shall be stored as MemoryVolatile.SelfTestResults to be retrieved by PTA or sent to the CGM to update the Car Network information.
3. VAP1.1.4 All failures detected during a self-test shall be logged in the MemoryNonVolatile.EventLogFile.
4. VAP1.1.5 All memories failures shall reset the DCM Application.

d) Performance
Not Applicable

e) Degraded Mode
Not Applicable

3.2.1.2 VAP1.3 Detect events

a) Purpose
DCM application should identify Infos and Failures based on the input signal values or by the system self-diagnostic. Events are project's specific and defined in the FMP.

b) Data

Source	Input
Input sense points	All inputs signal values listed in section 2.1.1

Destination	Output
Memory	MemoryNonVolatile.EventStatus
Memory	MemoryNonVolatile.EventLogFile
Memory	MemoryVolatile.MODE.DEGRADED.DisableOpen
Memory	MemoryVolatile.MODE.DEGRADED.DisableClose
Memory	MemoryVolatile.MODE.DEGRADED.ReducedSpeed

c) Processing

1. VAP1.3.1 The DCM application shall monitor all inputs signal values listed in section 2.1.1 and system components in order to identify any unusual Info, Exception or Failure as defined in the project's specific Fault Management Plan (FMP).
2. VAP1.3.2 If an Info or Failure is found, the MemoryNonVolatile.EventStatus shall be updated accordingly and it shall be logged in MemoryNonVolatile.EventLogFile.
3. VAP1.3.3 If an Info or Failure disappears, the MemoryNonVolatile.EventStatus shall be updated accordingly and it shall be logged in MemoryNonVolatile.EventLogFile.
4. VAP1.3.4 The DCU Application shall set the flag MemoryVolatile.MODE_DEGRADED.DisableOpen when any one of "set" Events has attribute of "Disable Open". The DCU Application shall reset the flag MemoryVolatile.MODE_DEGRADED.DisableOpen when there are no "set" Events with "Disable Open" attribute.
5. VAP1.3.5 The DCU Application shall set the flag MemoryVolatile.MODE_DEGRADED.DisableClose when any one of "set" Events has attribute of "Disable Close". The DCU Application shall reset the flag MemoryVolatile.MODE_DEGRADED.DisableClose when there are no "set" Events with "Disable Close" attribute.

6. VAP1.3.6 The DCU Application shall set the flag MemoryVolatile.MODE_DEGRADED. ReducedSpeed when any one of "set" Events has attribute of "Reduced Speed". The DCU Application shall reset the flag MemoryVolatile.MODE_DEGRADED.ReducedSpeed when there are no "set" Events with "Reduced Speed" attribute.

d) Performance
Not Applicable

e) Degraded Mode
Not Applicable

3.2.2 Door operator movements functions

3.2.2.1 VAP1.4 Door open sequence

a) Purpose
The Door open sequence can optionally be initiated either by trainlines or by a crew key (one door panel of a car side).
See Section 3.2.1.2 and the SACS document for the requirements of the setting and resetting flags in the MODE.DEGRADED.

b) Data

Source	Input
Input sense points	Input.MotorTypeID.ISO
Input sense points	Input.DO.MotorTemp.ISO
Configuration file	SACS.Door.OpeningTime
Configuration file	SACS.Door.OpeningTime.Range

Destination	Output
Outputs	Door open sequence
Outputs	Low current limit Error
Memory	MemoryVolatile.MODE.DEGRADED.DisableOpen
Memory	MemoryVolatile.MODE.DEGRADED.ReducedSpeed

c) Processing
Door command:

1. VAP1.4.1 Upon activation of a valid open command, DCM application shall initiate the door open sequence corresponding to the Input.MotorTypeID.ISO.

2. VAP1.4.18 To open the door using the Outside Crew Switch, the DCM Application shall detect a valid Outside Crew Switch Open command.

Optional:

3. VAP1.4.19 To open the door using the Inside Crew Switch, the DCM Application shall detect shall detect a valid Inside Crew Switch Open command..

Door control:

4. VAP1.4.13 The door shall open in the defined time (SACS.Door.OpeningTime) by default.

Optional:

5. VAP1.4.4 If Input.DO.MotorTemp is measured and over the temperature limit, MemoryVolatile. MODE_DEGRADED.ReduceSpeed shall be set and the low current limit Error shall be activated.

Optional:

6. VAP1.4.5 If a solenoid lock is present, the DCM application shall first drive the motor in close direction to remove any pressure and unlock the door by driving the solenoid lock.

7. VAP1.4.12 The door shall be maintained in the fully open position after opening.

8. VAP1.4.17 The MemoryNonVolatile.DoorMovementinformationFiles for opening shall include door speed, PWM motor command, and position error.

d) Performance

9. VAP1.4.14 The DCM Application shall open doors within tolerance of ±100ms from the configured opening time (SACS.Door.OpeningTime).

e) Degraded Mode

10. VAP1.4.15 The DCM Application shall open doors with maximum opening time (SACS.Door.OpeningTime.Range) when the MemoryVolatile.MODE_DEGRADED. ReduceSpeed is set.

11. VAP1.4.16 The DCM Application shall not open doors when the MemoryVolatile.MODE_DEGRADED.DisableOpen flag is set

3.2.2.2 VAP1.5 Door close sequence

a) Purpose

The door close sequence can be initiated either by the trainlines or by a crew key.

b) Data

Source	Input
Input sense points	Input.MotorTypeID.ISO
Input sense points	Input.DO.MotorTemp
Memory	MemoryVolatile.PositionEncoderCount
Configuration file	SACS.Door.ClosingTime
Configuration file	SACS.Door.ClosingTime.Range
Configuration file	SACS.Door.ClosingDelay
Configuration file	SACS.Door.VisualAlarmActivation
Configuration file	SACS.Door.AudibleAlarmActivation
Input sense points	Input.DO.LockSSR.ISO
Input sense points	Input.DO.LockSSL.ISO

Destination	Output
Outputs	Door close sequence
Memory	MemoryVolatile.MODE.DEGRADED.DisableClose
Memory	MemoryVolatile.MODE.DEGRADED.ReducedSpeed
Memory	MemoryNonVolatile.SystemCountersFile
Outputs	Output.PassAlertLight.ISO
Outputs	Output.PassAudibleSignal.ISO

c) Processing

Door command:

1. VAP1.5.1 When the door is not already closed and locked, and the DCM receive a valid close command; the DCM application shall activate the door closing warning indicator for the SACS.Door.ClosingDelay delay and initiate the door close sequence corresponding to the Input.MotorTypeID.ISO.

2. VAP1.5.15 To close the door using the Outside Crew Switch, the DCM Application shall detect a valid Outside Crew Switch Close command.

Optional:

3. VAP1.5.16 To close the door using the Inside Crew Switch, the DCM Application shall detect a valid Inside Crew Switch Close command.

Passenger Alerting Light and Passenger Audible Signal:

4. VAP1.5.18 The DCM Application shall flash the Output.PassAlertLight.ISO (If SACS.Door.VisualAlarmActivation is activated) and Output.PassAudibleSignal.ISO (If SACS.Door.AudibleAlarmActivation is activated) only when a valid Close command is detected.

Door control:

5. VAP1.5.4 The door shall close according to configured time (SACS.Door.ClosingTime).
6. VAP1.5.14 The MemoryNonVolatile.DoorMovementinformationFiles for closing shall include door speed, PWM motor command, and position error.

Door cycle counter:

1. VAP1.5.17 Upon closure of the door, the DCM shall increment the door cycle counter within the MemoryNonVolatile.SystemCountersFile.

d) Performance

2. VAP1.5.11 The DCM application shall close doors within tolerance of ±100ms from the configured closing time (SACS.Door.ClosingTime).

e) Degraded Mode

3. VAP1.5.2 If the door has never been locked since last power-up, MemoryVolatile.PositionEncoderCount shall be considered unknown and the door shall close slowly, in seek-home mode.
4. VAP1.5.12 The DCM Application shall close doors using the maximum closing time (SACS.Door.ClosingTime.Range) when the MODE.DEGRADED flag “Reduced Speed” is set.
5. VAP1.5.13 The DCM Application shall not close doors when the MODE.DEGRADED flag “Disable Close” is set

Optional:

6. VAP1.5.3 If Input.DO.MotorTemp is over the limit, the MODE.DEGRADED flag “Reduced Speed” shall be set to minimize the load on the motor that is already overheating.

3.2.2.3 VAP1.6 Detect obstruction

a) Purpose

The obstruction detection is configurable to be active during door close sequence and is used to prevent closure of a door if there is an object in its way.

b) Data

Source	Input
Input sense points	Input.DO.MotorEncPhaseA
Input sense points	Input.DO.MotorEncPhaseB
Input sense points	Input.MotorTypeID.ISO
Outputs	MemoryVolatile.PositionEncoderCoun t
Configuration file	SACS.Door.Obstruction.Distance
Configuration file	SACS.Door.Obstruction.Count
Configuration file	SACS.Door.Obstruction.EndBehavior

Destination	Output
Outputs	Door obstruction sequence

c) Processing

1. VAP1.6.1 DCM application shall detect a flat bar, 3/8-inch (9.53 mm) thick and 3 inches (76.2 mm) wide, held rigidly between and perpendicular to the door panel, as a hand might be held to stop the doors.
2. VAP1.6.2 This sensitivity shall be required throughout the entire length of the panel and for both values of the Input.MotorTypeID.ISO.
3. VAP1.6.3 It shall also detect an object, 7/16-inch (11.11 mm) in diameter, held rigidly between and perpendicular to the door panels at all locations along the length of the door nosing seal for all motor types supported, except the uppermost and lowermost 4 inch (101.6 mm) of the seal.
4. VAP1.6.4 When DCM application is closing the door, if the DCM detects an obstruction i.e. a discrepancy between current MemoryVolatile.PositionEncoderCount (calculated by identifying the sequence of Input.DO.MotorEncPhaseA and Input.DO.MotorEncPhaseB values) and the calculated profile position, DCM application shall control the motor to stop, then if the door has not reached the defined obstruction retries limit (SACS.Door.Obstruction.Count), the door shall open by SACS.Door.Obstruction.Distance and retry to close.
5. VAP1.6.8 When the defined maximum number of retries (SACS.Door.Obstruction.Count) is exceeded, the DCM application shall follow the end behavior selected by SACS.Door.Obstruction.EndBehavior value into the list of 3 pre-defined behavior :
   - 0 Stand Still: the door does not move until the next received command.
   - 1 Fully-Reopen: The door fully reopen at the minimum speed.
   - 2 Force Close: The door keep driving in the close direction.
6. VAP1.6.10 When SACS.Door.Obstruction.EndBehavior is in Force Close mode, the current in the motor shall be limited to avoid temperature build up in the motor.

d) Performance

Not Applicable

e) Degraded Mode
Not Applicable

3.2.3 VAP1.44 Emergency Handle Switch Function

a) Purpose

The emergency door opening is made by pulling down the Emergency Handle. The handle is spring loaded, and it returns to its original position when released.

b) Data

Source	Input
Input sense points	Input.DO.EmerHandleSwitch.ISO

Destination	Output
Outputs	CPU.PWM.MOTOR

c) Processing

1. VAP1.44.1 When the Input.DO.EmerHandleSwitch.ISO is activated the DCU Application shall cut the motor power (CPU.PWM.MOTOR), causing the door to be free to move manually.

d) Performance

Not Applicable

e) Degraded Mode

Not Applicable

3.2.4 VAP1.45 Door Cutout Function

a) Purpose

When the Door Cutout is activated on a door panel, the door cannot be opened and is isolated from the rest of the train.

b) Data

Source	Input
Input sense points	Input.DPCutoutSwitchR.ISO
Input sense points	Input.DPCutoutSwitchL.ISO

Destination	Output
Outputs	Output.DO.SolenoidDrv

c) Processing

1. VAP1.45.1 When Input.DPCutoutSwitchR.ISO or Input.DPCutoutSwitchL.ISO is activated the DCM Application shall keep the corresponding door locked (Output.DO.SolenoidDrv = Off) and ignore further command until the Cutout switch is returned to its OFF position.

d) Performance

Not Applicable

e) Degraded Mode

Not Applicable

3.2.5 VAP1.8 Manage Event log

a) Purpose

The DCM application should maintain an internal file for recording Infos. Only Failures and Exceptions should be accessible from the MDS. The PTA should access Infos, Failures and Exceptions.

b) Data

Source	Input
Memory	MemoryVolatile.TemporaryData

Destination	Output
Memory	MemoryNonVolatile.EventLogFile
Memory	MemoryVolatile.TemporaryData
Memory	MemoryNonVolatile.DoorMovementInformationFiles

c) Processing

1. VAP1.8.1 Upon detection of an event, the DCM application shall store the specific event information along with a timestamp in the MemoryNonVolatile.EventLogFile to prevent information loss resulting from a power failure.
2. VAP1.8.2 The MemoryNonVolatile.EventLogFile shall be able to store 1500 entries.
3. VAP1.8.3 A maximum of 10 entries per event number shall be stored for a single day.
4. VAP1.8.4 In case of an overflow of the MemoryNonVolatile.EventLogFile, the new data entries shall replace the entries that are stored at the beginning of the MemoryNonVolatile.EventLogFile.

d) Performance

Not Applicable

e) Degraded Mode

Not Applicable

3.2.6 VAP1.42 Manage Snapshot Files

a) Purpose

DCM application should maintain internal files that will contain snapshots. The data is available to the PTA upon request and it is intended to be used in conjunction with the event log file to determine which signals are leading to an event.
Each set of snapshot samples (SACS.Door.Snapshot.Capacity + SACS.Door.Snapshot. SampleNbAfterevent + 1) is considered as one snapshot file, i.e. each entry is the set of individual snapshot samples that are to be associated with the specified event. The number of MemoryNonVolatile.SnapshotFile should be determined in conjunction with the memory required by the MemoryNonVolatile.EventLogFile.

b) Data

Source	Input
Memory	MemoryVolatile.TemporaryData
Configuration file	SACS.Door.Snapshot.Period
Configuration file	SACS.Door.Snapshot.Capacity
Configuration file	SACS.Door.Snapshot.SampleNbAfterEvent

Destination	Output
Memory	MemoryNonVolatile.SnapshotFile
Memory	MemoryVolatile.TemporaryData
Memory	MemoryNonVolatile.DoorMovementinformationFiles

c) Processing

1. VAP1.42.1 Each snapshot sample is a single set of values for a specified group of signals, a snapshot sample shall be taken every SACS.Door.Snapshot.Period.

2. VAP1.42.2 The SACS.Door.Snapshot.Capacity most recent snapshot samples are stored in a circular buffer in the MemoryVolatile.SnapshotBuffer to keep the last (SACS.Door.Snapshot.Capacity * SACS.Door.Snapshot.Period)/1000 seconds status.

3. VAP1.42.3 When a event with the snapshot attribute occurs, the set of snapshot samples located in the MemoryVolatile.TemporaryData, the next SACS.Door.Snapshot. SampleNbAfterevent snapshots, the current sample and the information identifying the event shall be stored as a MemoryNonVolatile.SnapshotFile.

3.2.7 PTA communication functions

3.2.7.1 VAP1.35 Check incoming and outgoing messages validity

a) Purpose
The DCM application has to check the validity of every message incoming from the PTA and should check the format of every outgoing message it sends.

b) Data

Source	Input
HW_COM	HW_COM.PTA.Message
HW_COM	MemoryVolatile.LoginState
HW_COM	Input.DeviceID

Destination	Output
HW_COM	Function call following HW_COM.PTA.Message Verification
HW_COM	HW_COM.DMS.Message

c) Processing

1. VAP1.35.1 Upon reception of HW_COM.PTA.Message, the DCM application shall verify the integrity of the message content, that the MemoryVolatile.LoginState is connected and that the Input.DeviceID of the message corresponds to the Controller ID of the CPU. If HW_COM.PTA.Message fails one or more of the validity tests, HW_COM.PTA.Message should not be taken into account by the DCM application.
2. VAP1.35.3 Before sending HW_COM.DMS.Message, the DCM shall include the integrity check result and the Input.DeviceID value of its CPU into the message.

d) Performance
Not Applicable

e) Degraded Mode
Not Applicable

3.2.7.2 VAP1.9 Login

a) Purpose
The DCM application should not accept any PTA command before the PTA is logged in.

b) Data

Source	Input
HW_COM	HW_COM.PTA.LoginRequest

Destination	Output
HW_COM	HW_COM.DMS.LoginResponse
Memory	MemoryVolatile.LoginState

c) Processing

1. VAP1.9.1 Upon reception of the HW_COM.PTA.LoginRequest from the PTA, the DCM application shall send HW_COM.DMS.LoginResponse, set MemoryVolatile.LoginState to Connected and start accepting other PTA commands.

d) Performance
Not Applicable

e) Degraded Mode
Not Applicable

3.2.7.3 VAP1.10 Logout / reboot

a) Purpose
The DCM application should logout upon command from the PTA via HW_COM. The DCM application should reboot upon request from the PTA.

b) Data

Source	Input
HW_COM	HW_COM.PTA.LogoutRequest
HW_COM	HW_COM.PTA.RebootRequest

Destination	Output
HW_COM	HW_COM.DMS.LogoutResponse
Memory	MemoryVolatile.LoginState

c) Processing

1. VAP1.10.3 Upon reception of the HW_COM.PTA.LogoutRequest command, the DCM application shall send HW_COM.DMS.LogoutResponse, set the MemoryVolatile. LoginState to Disconnected and wait for the PTA to login to accept any other PTA command.
2. VAP1.10.2 Upon reception of the HW_COM.PTA.RebootRequest command or a watchdog reset, the DCM application shall discontinue all operations and perform a reboot.

d) Performance
Not Applicable

e) Degraded Mode
Not Applicable

3.2.7.4 VAP1.41 Automatic logout

a) Purpose
The purpose of this function is to automatically logout if no valid communication is received from the PTA for more than 1 minute.

b) Data

Source	Input
HW_COM	HW_COM.PTA.Message
HW_COM	HW_COM.PTA.RebootRequest

Destination	Output
HW_COM	MemoryVolatile.LoginState

c) Processing

1. VAP1.41.1 Upon no reception of HW_COM.PTA.Message for more than 1 minute while MemoryVolatile.LoginState = Connected, the DCM shall cease communication with the PTA by logging out (MemoryVolatile.LoginState = Disconnected).

d) Performance
Not Applicable

e) Degraded Mode
Not Applicable

3.2.7.5 VAP1.39 Verify connection state

a) Purpose
The purpose of this function is to test the state of connection periodically.

b) Data

Source	Input
HW_COM	HW_COM.PTA.HeartbeatRequest

Destination	Output
HW_COM	HW_COM.DCM.HeartbeatResponse

c) Processing

1. VAP1.39.1 Every 30 seconds, the PTA send HW_COM.PTA.HeartbeatRequest to the DCM and the DCM shall send HW_COM.DMS.HeartbeatResponse to the PTA in answer, to ensure that a working connection is still established between the DCM and the PTA.

d) Performance
Not Applicable

e) Degraded Mode
Not Applicable

3.2.7.6 VAP1.11 Send location identification

a) Purpose
The DCM application should send its location identification to the PTA.

b) Data

Source	Input
HW_COM	HW_COM.PTA.LocationIdentification Request
Memory	MemoryVolatile.LocationIdentification

Destination	Output
HW_COM	HW_COM.DMS.LocationIdentificatio nResponse

c) Processing

1. VAP1.11.1 Upon reception of the HW_COM.PTA.LocationIdentificationRequest from the PTA, the DCM application shall send HW_COM.DMS.LocationIdentificationResponse message to the PTA containing MemoryVolatile.LocationIdentification.

d) Performance
Not Applicable

e) Degraded Mode
Not Applicable

3.2.7.7 VAP1.37 Send car number

a) Purpose
The DCM application should send its car number to the PTA.

b) Data

Source	Input
HW_COM	HW_COM.PTA.CarNumberRequest
Memory	MemoryVolatile.CarNumber

Destination	Output
HW_COM	HW_COM.DMS.CarNumberRespon se

c) Processing

1. VAP1.37.1 Upon reception of the HW_COM.PTA.CarNumberRequest from the PTA, the DCM application shall send HW_COM.DMS.CarNumberResponse message to the PTA containing MemoryVolatile.CarNumber.

d) Performance
Not Applicable

e) Degraded Mode
Not Applicable

3.2.7.8 VAP1.12 Send time

a) Purpose
The DCM application should receive the local time request from the PTA and send its local time value to the PTA.

b) Data

Source	Input
HW_COM	HW_COM.PTA.LocalTimeRequest
Memory	MemoryVolatile.LocalTime

Destination	Output
HW_COM	HW_COM.DMS.LocalTimeRespons e

c) Processing

1. VAP1.12.1 Upon reception of the HW_COM.PTA.LocalTimeRequest from the PTA, the DCM application shall send HW_COM.DMS.LocalTimeResponse containing the current MemoryVolatile.LocalTime value to the PTA.

d) Performance
Not Applicable

e) Degraded Mode
Not Applicable

3.2.8 VAP1.43 Set time

a) Purpose
The DCM application should receive the set local time request from the PTA and change its local time value accordingly.

b) Data

Source	Input
HW_COM	HW_COM.PTA.SetTimeRequest

Destination	Output
HW_COM	HW_COM.DMS.SetTimeResponse
Memory	MemoryVolatile.LocalTime

c) Processing

1. VAP1.43.1 Upon reception of the HW_COM.PTA.SetTimeRequest from the PTA, the DCM application shall send HW_COM.DMS.SetTimeResponse and change MemoryVolatile.LocalTime with the value received from the PTA.

d) Performance
Not Applicable

e) Degraded Mode
Not Applicable

3.2.8.1 VAP1.13 Send self-test results

a) Purpose
The DCM application should send self-tests result to the PTA.

b) Data

Source	Input
HW_COM	HW_COM.PTA.SelfTestRequest

Destination	Output
HW_COM	HW_COM.DMS.SelfTestResponse

c) Processing

1. VAP1.13.1 Upon reception of the HW_COM.PTA.SelfTestRequest from the PTA, the DCM application shall send HW_COM.DMS.SelfTestResponse to the PTA with last test results.

d) Performance
Not Applicable

e) Degraded Mode
Not Applicable

3.2.8.2 VAP1.14 Send code status

a) Purpose
The DCM application should send code size and integrity check result of DCM application and Boot Loader application to the PTA.

b) Data

Source	Input
HW_COM	HW_COM.PTA.CodeStatusRequest
Memory	MemoryNonVolatile.DCMApplicationIntegrityCheck
Memory	MemoryNonVolatile.BootLoaderApplicationIntegrityCheck
Memory	MemoryNonVolatile.DCMApplicationCodeSize
Memory	MemoryNonVolatile.BootLoaderApplicationCodeSize

Destination	Output
HW_COM	HW_COM.DMS.CodeStatusResponse

c) Processing

1. VAP1.14.1 Upon reception of the HW_COM.PTA.CodeStatusRequest from the PTA, the DCM application shall send HW_COM.DMS.CodeStatusResponse containing MemoryNonVolatile.DCMApplicationIntegrityCheck, MemoryNonVolatile.BootLoaderApplicationIntegrityCheck, MemoryNonVolatile.DCMApplicationCodeSize, MemoryNonVolatile.BootLoaderApplicationCodeSize to the PTA.

d) Performance
Not Applicable

e) Degraded Mode
Not Applicable

3.2.8.3 VAP1.15 Update PTA real time monitoring data

a) Purpose
The DCM application should send all sense point values to the PTA to perform real time monitoring upon request from the PTA.

b) Data

Source	Input
HW_COM	HW_COM.PTA.MonitoringRequest
Input sense points	All inputs signal values listed in section 2.1.1
Output Read back	All outputs signal values listed in section 2.1.1

Destination	Output
HW_COM	HW_COM.DMS.MonitoringRespons e

c) Processing

1. VAP1.15.1 Upon reception of HW_COM.PTA.MonitoringRequest from the PTA, the DCM application shall send HW_COM.DMS.MonitoringResponse, transmitting all inputs signal values and all outputs signal values listed in section 2.1.1 and defined in the OPDF.

d) Performance
Not Applicable

e) Degraded Mode
Not Applicable

3.2.8.4 VAP1.16 Update event status

a) Purpose
The DCM application should send current event status to the PTA upon request.

b) Data

Source	Input
HW_COM	HW_COM.PTA.EventStatusRequest
Memory	MemoryNonVolatile.EventStatus

Destination	Output
HW_COM	HW_COM.DMS.EventStatusRespon se

c) Processing

1. VAP1.16.1 Upon reception of the HW_COM.PTA.EventStatusRequest from the PTA, the DCM application shall send HW_COM.DMS.EventStatusResponse containing the MemoryNonVolatile.EventStatus as defined in the EDF.

d) Performance
Not Applicable

e) Degraded Mode
Not Applicable

3.2.8.5 VAP1.17 Erase event log file

a) Purpose
The DCM application should erase the event log file upon request from the PTA via HW_COM.

b) Data

Source	Input
HW_COM	HW_COM.PTA.EraseEventLogFileRequest

Destination	Output
Memory	MemoryNonVolatile.EventLogFile
HW_COM	HW_COM.DMS.EraseEventLogFileResponse

c) Processing

1. VAP1.17.1 Upon reception of HW_COM.PTA.EraseEventLogFileRequest from the PTA, the DCM application shall erase all entries in MemoryNonVolatile.EventLogFile and send HW_COM.DMS.EraseEventLogFileResponse.

d) Performance
Not Applicable

e) Degraded Mode
Not Applicable

3.2.8.6 VAP1.18 Erase snapshot file

a) Purpose
The DCM application should erase a selected snapshot file upon request from the PTA via HW_COM.

b) Data

Source	Input
HW_COM	HW_COM.PTA.EraseSnapshotFileRequest

Destination	Output
Memory	MemoryNonVolatile.SnapshotFile
HW_COM	HW_COM.DMS.EraseSnapshotFileResponse

c) Processing

1. VAP1.18.1 Upon reception of HW_COM.PTA.EraseSnapshotFileRequest with a snapshot file index argument from the PTA, the DCM application shall erase the selected snapshot file in MemoryNonVolatile.SnapshotFile and send HW_COM.DMS.EraseSnapshotFileResponse.

d) Performance
Not Applicable

e) Degraded Mode
Not Applicable

3.2.8.7 VAP1.20 Transfer event log file

a) Purpose
The DCM application should transfer the event log file, which includes all Events to the PTA via HW_COM.

b) Data

Source	Input
HW_COM	HW_COM.PTA.EventLogFileRequest
Memory	MemoryNonVolatile.EventLogFile

Destination	Output
HW_COM	HW_COM.DMS.EventLogFileResponse

c) Processing

1. VAP1.20.1 Upon reception of HW_COM.PTA.EventLogFileRequest from the PTA, the DCM application shall send HW_COM.DMS.EventLogFileResponse containing all active entries of MemoryNonVolatile.EventLogFile to the PTA.

d) Performance
Not Applicable

e) Degraded Mode
Not Applicable

3.2.8.8 VAP1.21 Transfer snapshot file

a) Purpose
The DCM application should transfer snapshot files to the PTA via HW_COM.

b) Data

Source	Input
HW_COM	HW_COM.PTA.SnapshotFileRequest
Memory	MemoryNonVolatile.SnapshotFile

Destination	Output
HW_COM	HW_COM.DMS.SnapshotFileResponse

c) Processing

1. VAP1.21.1 Upon reception of HW_COM.PTA.SnapshotFileRequest from the PTA, the DCM application shall send HW_COM.DMS.SnapshotFileResponse to the PTA, containing MemoryNonVolatile.SnapshotFile.

d) Performance
Not Applicable

e) Degraded Mode
Not Applicable

3.2.8.9 VAP1.22 Transfer event description file and operating parameters description file

a) Purpose
The DCM application should transfer the event description file and operating parameters description file to the PTA via HW_COM.

b) Data

Source	Input
HW_COM	HW_COM.PTA.EventDescriptionFileRequest
HW_COM	HW_COM.PTA.OperatingParametersDescriptionFileRequest
Memory	MemoryNonVolatile.EventDescriptionFile
Memory	MemoryNonVolatile.OperatingParametersDescriptionFile

Destination	Output
HW_COM	HW_COM.DMS.EventDescriptionFileResponse
HW_COM	HW_COM.DMS.OperatingParametersDescriptionFileResponse

c) Processing

1. VAP1.22.1 Upon reception of the HW_COM.PTA.EventDescriptionFileRequest from the PTA, the DCM application shall send HW_COM.DMS.EventDescriptionFileResponse containing the MemoryNonVolatile.EventDescriptionFile.

2. VAP1.22.2 Upon reception of the HW_COM.PTA.OperatingParametersDescriptionFileRequest, the DCM application shall send HW_COM.DMS.OperatingParametersDescriptionFileResponse containing the MemoryNonVolatile.OperatingParametersDescriptionFile.

d) Performance
Not Applicable

e) Degraded Mode
Not Applicable

3.2.8.10 VAP1.23 Transfer door movement information file

a) Purpose
The DCM application should transfer door movement information file to the PTA via HW_COM.

b) Data

Source	Input
HW_COM	HW_COM.PTA.DoorMovementInformationFilesRequest
Memory	MemoryNonVolatile.DoorMovementInformationFiles

Destination	Output
HW_COM	HW_COM.DMS.DoorMovementInformationFilesResponse

c) Processing

1. VAP1.23.1 Upon reception HW_COM.PTA.DoorMovementInformationFilesRequest, the DCM application shall send HW_COM.DMS.DoorMovementInformationFilesResponse containing MemoryNonVolatile.DoorMovementInformationFiles.

d) Performance
Not Applicable

e) Degraded Mode
Not Applicable

3.2.8.11 VAP1.24 Transfer software versions

a) Purpose
The DCM application should transfer software versions to the PTA via HW_COM.

b) Data

Source	Input
HW_COM	HW_COM.PTA.SoftwareVersionsRequest
Memory	MemoryNonVolatile.DCMApplicationVersion
Memory	MemoryNonVolatile.BootLoaderApplicationVersion

Destination	Output
HW_COM	HW_COM.DMS.SoftwareVersionsResponse

c) Processing

1. VAP1.24.1 Upon reception of a HW_COM.PTA.SoftwareVersionsRequest, the DCM application shall send HW_COM.DMS.SoftwareVersionsResponse containing the MemoryNonVolatile.DCMApplicationVersion, and MemoryNonVolatile.BootLoaderApplicationVersion.

d) Performance
Not Applicable

e) Degraded Mode
Not Applicable

3.2.8.12 VAP1.40 Transmit DCM memory dump

a) Purpose
The DCM provides the function to download and view the DCM volatile memory contents from the PTA, as well as to set the beginning and the end of the memory zone to download.

b) Data

Source	Input
HW_COM	HW_COM.PTA.MemoryDumpRequest
Memory	MemoryVolatile.Content

Destination	Output
HW_COM	HW_COM.DMS.MemoryDumpResponse

c) Processing

1. VAP1.40.1 Upon reception of HW_COM.PTA.MemoryDumpRequest from the PTA, the DCM shall send HW_COM.DMS.MemoryDumpResponse in return, containing the MemoryVolatile.Content portion defined by the starting address and the end address of the memory zone to download transmitted by the PTA.

d) Performance
Not Applicable

e) Degraded Mode
Not Applicable

3.2.8.13 VAP1.25 Read door parameters

a) Purpose
The DCM application should send its parameters upon PTA request via HW_COM.

b) Data

Source	Input
HW_COM	HW_COM.PTA.ReadDMSConfigurationRequest
Configuration file	All parameters listed in Appendix A

Destination	Output
HW_COM	HW_COM.DMS.ReadDMSConfigurationResponse

c) Processing

1. VAP1.25.1 Upon reception of the HW_COM.PTA.ReadDMSConfigurationRequest, the DCM application shall send HW_COM.DMS.DMSConfigurationResponse containing all configurable parameters listed in Appendix A.

2. VAP1.25.2 Available parameters and their read/write associated permissions shall be as defined in the Appendix A.

d) Performance
Not Applicable

e) Degraded Mode
Not Applicable

3.2.8.14 VAP1.36 Write door parameters

a) Purpose
The DCM application should modify its parameters upon PTA request via HW_COM.

b) Data

Source	Input
HW_COM	HW_COM.PTA.WriteDMSConfigurationRequest
Configuration File	SACS.Door.OpeningTime.Range
Configuration File	SACS.Door.ClosingTime.Range
Configuration File	SACS.Door.Obstruction.Range

Destination	Output
HW_COM	HW_COM.DMS.WriteDMSConfigurationResponse

c) Processing

1. VAP1.36.1 Upon reception of the HW_COM.PTA.WriteDMSConfigurationRequest, the DCM application shall update the user configurable parameters in the Configuration File (See Appendix A for description of each parameter) and send HW_COM.DMS.WriteDMSConfigurationResponse.
2. VAP1.36.3 Opening time shall be adjustable within the defined range (SACS.Door.OpeningTime.Range) for all motor types supported.
3. VAP1.36.4 Closing time shall be adjustable into the defined range (SACS.Door.ClosingTime.Range) for both motor types supported.
4. VAP1.36.5 Obstruction retries shall be adjustable within the defined range (SACS.Door.Obstruction.Range).

d) Performance
Not Applicable

e) Degraded Mode
Not Applicable

3.2.8.15 VAP1.31 Reset system counters

a) Purpose
The DCM application should reset its internal system counters files values upon request from the PTA. The counters saved by the system are described in Section 3.3.7.

b) Data

Source	Input
HW_COM	HW_COM.PTA.SystemCountersResetRequest
Memory	MemoryNonVolatile.SystemCountersFile

Destination	Output
Memory	HW_COM.DMS.SystemCountersResetResponse

c) Processing

1. VAP1.31.5 Upon reception of the HW_COM.PTA.SystemCountersResetRequest, the DCM application shall reset MemoryNonVolatile.SystemCountersFile and update the counters related elements and then send HW_COM.DMS.SystemCountersReset Response

d) Performance
Not Applicable

e) Degraded Mode
Not Applicable

3.2.8.16 VAP1.34 Transfer system counter file

a) Purpose
The DCM application should transfer system counters files to the PTA via HW_COM.

b) Data

Source	Input
HW_COM	HW_COM.PTA.SystemCountersRequest
Memory	MemoryNonVolatile.SystemCountersFile

Destination	Output
Memory	HW_COM.DMS.SystemCountersResponse

c) Processing

1. VAP1.34.1 Upon reception of HW_COM.PTA.SystemCountersRequest, the DCM application shall send HW_COM.DMS.SystemCountersResponse containing Memory NonVolatile.SystemCountersFile.

d) Performance
Not Applicable

e) Degraded Mode
Not Applicable

3.2.8.17 VAP1.38 Transfer DCM memory dump

a) Purpose
The DCM application should transfer the volatile memory contents upon request from the PTA.

b) Data

Source	Input
HW_COM	HW_COM.PTA.MemoryDumpRequest
Memory	MemoryVolatile.Content

Destination	Output
HW_COM	HW_COM.DMS.MemoryDumpResponse

c) Processing

1. VAP1.38.1 Upon reception of HW_COM.PTA.MemoryDumpRequest from the PTA, the DCM application shall send HW_COM.DMS.MemoryDumpResponse containing MemoryVolatile.Content according to requested range.

d) Performance
Not Applicable

e) Degraded Mode
Not Applicable

VAP1.27 Performance requirements

This section identifies the static and dynamic numerical requirements made on software.

3.2.9 VAP1.27.1 Static numerical requirements

1. VAP1.27.1.2 The DCU Application shall use less than 90% of Volatile memory.
2. VAP1.27.1.4 The DCU Application shall use less than 45% of the Non-Volatile memory.

3.2.10 VAP1.27.2 Dynamic numerical requirements

1. VAP1.27.2.2 DCM application shall use less than 50% of available processing time.

3.3 VAP1.32 Logical database requirements

3.3.1 VAP1.32.1 Event log file

1. VAP1.32.1.1 The MemoryNonVolatile.EventLogFile shall contain 1500 events.
2. VAP1.32.1.2 The MemoryNonVolatile.EventLogFile shall be a circular buffer (after the 1500th event is logged, the 1501st shall overwrite the first entry).
3. VAP1.32.1.4 Each event in the MemoryNonVolatile.EventLogFile shall contains the following data :
   - Timestamp
   - Event code (event number)
   - 16 attributes of each event (bit field)
   - System status (for timestamp validity)
   - Integrity check result for each event.

3.3.2 VAP1.32.2 Snapshot file

1. VAP1.32.2.1 The DCM application shall be able to capture SACS.Door.Snapshot.NbOfFiles snapshot files.
2. VAP1.32.2.2 The DCM application shall be able to store up to SACS.Door.Snapshot.NbOfFiles snapshot files when its panel is mechanically cut-out.
3. VAP1.32.2.3 Each snapshot file shall contain (SACS.Door.Snapshot.Capacity) samples of the operating parameters status. Sample shall be taken every 1000/SACS.Door.Snapshot.Period milliseconds: SACS.Door.Snapshot.Capacity/2 samples taken before the event, 1 at event time, and SACS.Door.Snapshot.Capacity/2 after the event occurs.
4. VAP1.32.2.7 Each MemoryNonVolatile.SnapshotFile stored in DCM shall contain the data displayed in Table 3-25.

Table 3-25: Snapshot Record Structure

```
- The “UInt16Type “ type represent an unsigned integer 16 bit size.

#define  MAX_LDTS_SNAPSHOT_PARAMETERS       100

// This is the Snapshot file header.
typedef struct {
    UCharType    aLocation[LOCATION_LEN]; // Node’s LOCATION_ID string.
    UInt16Type   aEventCode;              // Reference event.
    Bytetype     aTimeStamp[4];           // Time at which the fault was triggered.
                                          // The time stamp is an unsigned 32 bits serial
                                          // number that represent the number of seconds elapsed
                                          // since 1 January 1970 (This time stamp structure supports
                                          // a wider range than the 1990 to 2100 required). The time
                                          // must be recorded in UTC. The TimeStamp shall also serve
                                          // as the name of the Snapshot file.
                                          // The TimeStamp is synchronized with the time received from
                                          // CDS.
    ByteType     aStoragePeriodMult;      // This value ranges from 1 to 255, representing the number
                                          // of 10 msec. increments. Default value is  25.
    ByteType     aNumberOfRecords;        // The number of records that follow the header.
    ByteType     aNumberOfParameters;     // The number of parameters in the file.
    UInt16Type   aIndexOfFirstValidRecord:// The index of the first record that contains valid data.
    ByteType     aMajorOpParamVersion;    //  Major  Version  of  the  Operating  parameters  Definition
File.
    ByteType     aMinorOpParamVersion;    //  Minor  Version  of  the  Operating  parameters  Definition
File.
    ByteType     aParameters[MAX_LDTS_SNAPSHOT_PARAMETERS];  // Parameter Id.
} Bbd_LdtsSnapShotHeaderStruct;

// This is the record saved in the LDTS snapshot file.
typedef struct {

     ByteType    aSignal0;                // Signal for parameter Id 0.
     ByteType    aSignal1;                // Signal for parameter Id 1.
     ByteType    aSignal2;                // Signal for parameter Id 2.
     ByteType    aSignal3;                // Signal for parameter Id 3.
    …
    …
    …
    …
     ByteType    aSignaln;        // Signal for parameter Id n. The value of n shall correspond to the
                                  // value of the aNumberOfParameters field.
} SnapRecordStruct;
```

- The value “*aStoragePeriodMult* * 10 msec.” indicates the time interval between stored records in the LDTS snapshot file.
- The *aNumberOfRecords* field indicates the number of records saved in the file.
- The *aNumberOfParameters* field is used to indicate the number of parameters in the *aParameters* field.
- The *aIndexOfFirstValidRecord* field is used to indicate the index of the first record that contains valid data. This field is particularly useful in the following scenario:
    - “One second after a “reboot”, an event, which has a Snapshot associated to it, has occurred. Hence, the “2 seconds of data recording preceding the fault occurrence” requirement cannot be met since only 1 second of data is available.”
- The *aParameters* field is used to indicate the Identifier of the parameter. Each field of the *aParameters* member corresponds to the signal fields in the snapshot file. An example would be to sample parameter Id: 1,5,8,12 when event 56 is in “fault” state. Empty fields shall have a value of zero “0”.

The header would be:

Table 3-26: Snapshot Record Header

```
SnapShotVar.aEvent = 56;                    // Reference event.
SnapShotVar.aTimeStamp = PresentTime;       // Time of the snapshot in unsigned 32 bits.
SnapShotVar. aStoragePeriodMult =  10;      // 100 msec. storage period.
SnapShotVar. aNumberOfRecords = 41;         // The number of records in the file.
SnapShotVar.aNumberOfParameters = 4;        // The number of parameters in each record.
AIndexOfFirstValidRecord = 10;              // The index of the first record that contains valid data.
SnapShotVar.aMajorOpParamVersion = 1;       // Major version of the Operating Data Definition File.
SnapShotVar.aMinorOpParamVersion = 0;       // Minor version of the Operating Data Definition File.

SnapShotVar.aParameters[0] = 1;             // Parameter Id for signal 0.
SnapShotVar.aParameters[1] = 5;             // Parameter Id for signal 1.
SnapShotVar.aParameters[2] = 8;             // Parameter Id for signal 2.
SnapShotVar.aParameters[3] = 12;            // Parameter Id for signal 3.
All other parameter Id are “Not Significant” // Set to zero ( 0 ).
```

3.3.3 VAP1.32.3 Operating parameters description file

1. VAP1.32.3.1 The MemoryNonVolatile.OperatingParametersDescriptionFile shall define the parameters that are retrievable from the PTA
2. VAP1.32.3.2 These parameters shall be retrieved through the use of snapshot or RTDM.
3. VAP1.32.3.3 The file shall contain records displayed in Table 3-27 .

Table 3-27: Operating Parameters Description File Structure

```
The “ByteType “ type represent an 8 bit size.

#define               MAX_PARAMETER_DESC    20  // Maximum size for the parameter description.
#define               UNIT_SIZE             13  // Unit size.

typedef struct {
       ByteType       aMajorOpParamVersion;      // Major version of the operational parameter file.
       ByteType       aMinorOpParamVersion;      // Minor version of the operational parameter file.
       ByteType       aReserved1;                // Reserved for future use.
       ByteType       aReserved2;                // Reserved for future use.
} Bbd_OpParameterHeaderStruct;

typedef struct {
       ByteType       aParameterCode;            // The Parameter ID for this subsystem.
       ByteType       aUnit[UNIT_SIZE];          // Description of unit on 13 characters. (ex: VDC,
                                                 // PSI,  etc.)
       ByteType       aFormat;                   // How to display the parameter value. Possible
                                                 // types are :
       // 0. Type unused.
       // 1. Unsigned1: 0 to  255.
       // 2. Signed1 : -128  to  127
       // 3. Unsigned Decimal1 : 0.0 to 25.5
       // 4. Signed Decimal1 : -12.8  to 12.7
       // 5. Unsigned Decimal 2 : 0.00 to 2.55
       // 6. Signed Decimal2 : -1.28  to 1.27
       // 7. Unsigned2: 0 to 510 . This format indicates that the value (in byte size)
       // will be multiplied by 2 to be in the range 0 to 510 (resolution of 2).
       // 8. Unsigned4: 0 to 1020. This format indicates that the value (in byte size )
       //     will be multiplied by a factor of 4 to be in the range 0 to 1020
       // (resolution of 4).
       // 9. Unsigned8: 0 to 2040. This format indicates that the value (in byte size)
       //     will be multiplied by a factor of 8 to be in the range 0 to 2040
       // (resolution of 8).
       // 10. Signed2: -256 to 254. This format indicates that the value (in byte size)
       //       will be multiplied by a factor of 2 to be in the range -256 to 254
       // (resolution of 2).
       // 11. Signed4: -512 to 508. This format indicates that the value (in byte size)
       //       will be multiplied by a factor of 4 to be in the range -512 to 508
       // (resolution of 4).
       // 12. Signed8: -1024 to 1016. This format indicates that the value (in byte size)
       //       will be multiplied by a factor of 8 to be in the range -1024 to 1016
       // (resolution of 8).
       // 13. Signed128: -16384 to 16256. This format indicates that the value (in byte
       // size)  will be multiplied by a factor of 128 to be in the range -16384 to 16256
```

```
        // (resolution of 128).
        // 14. OffSet60: -60 to +195. This format indicates that the value  (in byte size)
        //  will have an offset of -60. Example: a value of 10 will be converted to –50
        //       using this format.
        // 15. Boolean: 0 or 1. This value is used for Boolean values; it is set to 0 to
        //       indicate FALSE (or OFF) and to 1 to indicate TRUE (or
        //    ON).
        // 16. Unsigned5: 0 to 1275. This format indicates that the value (in byte size )
        //     will be multiplied by a factor of 5 to be in the range 0 to 1275
        // (resolution of 5).
        // 17. UnsignedQuarter: 0 to 63.75. This format indicates that the value (in byte
        // size )  will be divided by a factor of 4 to be in the range 0 to 63.75
        // (resolution of 0.25).

        UInt16Type      aSamplingPeriod;                    // Operating parameter internal
                                                            // sampling period. This value
                                                            // may range from 1 to 65535,
                                                            // representing  increments of 10ms.

        ByteType        aDescription[MAX_PARAMETER_DESC];
                        // Description of the parameter. Shall not use a
                        //NULL termination. If less than 20  characters are
                        // necessary to describe the event, the space shall
                        // be filled with blank characters.
 } OpParameterRecordStruct;
```

3.3.4 VAP1.32.4 Event description file

1. VAP1.32.4.1 The MemoryNonVolatile.EventDescriptionFile shall be composed of a header and a list of records. The EDF shall be part of the DCM application code and shall not be changeable without upgrading the application.
2. VAP1.32.4.2 The event codes assignation in the EDF shall begin with “1” and be declared in ascending order up to a maximum of 1999 .
3. VAP1.32.4.3 In order to maintain backward version compatibility, no event code shall be deleted or replaced
4. VAP1.32.4.4 Events that are no longer used in newer software versions will remain in the list, and new events are added to the end of the existing list
5. VAP1.32.4.5 The version of the EDF shall be incremented every time the EDF is updated in a released DCM application
6. VAP1.32.4.6 The EDF shall contain records displayed in Table 3-28 .

Table 3-28: Event Description File Structure

```
#define MAX_EVENT_DESC_SIZE                31

typedef struct {
        ByteType        aMajorEdfVersion;   // Major version of the EDF.
        ByteType        aMinorEdfVersion;   // Minor version of the EDF.
        ByteType        aReserved1;         // Reserved for future use.
        ByteType        aReserved2;         // Reserved for future use.

} EdfFileHeaderStruct;


typedef struct {
        UInt16Type      aEventCode;         // Event code.

        AttributeStruct aAttributes;        // Event code attributes

        ByteType        aType :1;           // This field indicates if this event is a:
                                            // 0 = Pulse Type.
                                            // 1 = Fault / Reset Type.

        ByteType        aPad :7;            // Bit padding.
```

```
        ByteType        aDescription[MAX_EVENT_DESC_SIZE];
                                        // Null terminated textual event
                                        //description of the associated event
                                        //code. Maximum string size of 30
                                        // characters + null termination.
} EdfRecordStruct;
```

3.3.5 VAP1.32.5 DCM Configuration File

1. VAP1.32.5.1 The Configuration File shall contain adjustable door parameters.
2. VAP1.32.5.2 The Configuration File format shall contain the following data :
   - Variables formatted as per Appendix A
   - Integrity check result for the Configuration File

3.3.6 VAP1.32.8 Door movement information file

The door movement information file is used to verify the performance of door control.

1. VAP1.32.8.1 There shall be one pair of MemoryNonVolatile.DoorMovementinformation Files ready for transfer to the PTA, empty if needed, at any time. The first one for the last opening sequence and the second one file for the last closing sequence .

2. VAP1.32.8.2 The door movement information file shall contain the data displayed in Table 3-29.

Table 3-29: Door Movement Information File Structure

```
The “ByteType“ type represents an 8 bit size.
The “UInt16Type” type represents an unsigned integer of 16 bit size.
#define MAX_LOG_PID         2000

typedef struct _LogStructGraph
{
        int             iError;         // Signed error on 16 bits for the position
        long            lPosition;      // Opening Position in pulses
        char            cPwm;           // Signed Percentage value of the PWM command.
                                        // -100% to +100%
        char            cCurrent;       // Current -25.6 to 25.4 A
} LogStructGraph;

typedef struct _LogStruct
{
    unsigned int    uiLastIndex;    // Value stored that contains the last valid index
                                    // for the door movement
    unsigned char   ucConfiguredTime; // Time configured for the current graph.
    unsigned char   ucRecordRate;   // Value that hold the periodicity of the record
                                    // ie: 5 ==> means 0.005 seconds interval between
                                    // 2 records
                                    // ie: 50 ==> means 0.05 seconds interval between
                                    // 2 records
    unsigned int    uiMaxError;     // Max Position error used to draw a line in the
                                    // graph
    LogStructGraph  Data[MAX_LOG_PID]; // Structure holding the current stored data
                                    // for the door movement
} LogStruct;
```

3.3.7 VAP1.32.10 System counters files

1. VAP1.32.10.1 The system counters files shall store permanent counters of the door cycles, running time and the event occurrences in MemoryNonVolatile, using the format displayed in Table 3-30.

Table 3-30: System Counters File Structure

```
typedef struct {
    unsigned int  uiNumberOfFaults;
    unsigned int  uiModificationFlag;
    unsigned int  uiSpareHeader1;
    unsigned int  uiSpareHeader2;
    unsigned long ulDoorCycles;                  // Door Cycles
    unsigned long ulLastDoorCycles;
    unsigned long ulLastDoorCyclesReset;
    unsigned long ulLastDoorCyclesResetWibu;
    unsigned long ulPrevDoorCyclesReset;
    unsigned long ulPrevDoorCyclesResetWibu;
    unsigned int  uiDoorCyclesCRC;
    unsigned long ulRunningTime;                 // Running Time
    unsigned long ulLastRunningTime;
    unsigned long ulLastRunningTimeReset;
    unsigned long ulLastRunningTimeResetWibu;
    unsigned long ulPrevRunningTimeReset;
    unsigned long ulPrevRunningTimeResetWibu;
    unsigned int  uiRunningTimeCRC;
    unsigned long FaultOccurrences[FAULT_MAX_NUMBER];      // Faults occurrences
    unsigned long ulLastFaultOccurrencesReset;
    unsigned long ulLastFaultOccurrencesResetWibu;
    unsigned long ulPrevFaultOccurrencesReset;
    unsigned long ulPrevFaultOccurrencesResetWibu;
    unsigned int  uiFaultOccurrencesCRC;
    unsigned long ulConfigChange;                // Config Change
    unsigned long ulLastConfigChange;
    unsigned long ulLastConfigChangeReset;
    unsigned long ulLastConfigChangeResetWibu;
    unsigned long ulPrevConfigChangeReset;
    unsigned long ulPrevConfigChangeResetWibu;
    unsigned int  uiConfigChangeCRC;
    unsigned long ulAppUpgrade;                  // Application Upgrade
    unsigned long ulLastAppUpgrade;
    unsigned long ulLastAppUpgradeReset;
    unsigned long ulLastAppUpgradeResetWibu;
    unsigned long ulPrevAppUpgradeReset;
    unsigned long ulPrevAppUpgradeResetWibu;
    unsigned int  uiAppUpgradeCRC;
    unsigned long ulBootUpgrade;                 // Boot Upgrade
    unsigned long ulLastBootUpgrade;
    unsigned long ulLastBootUpgradeReset;
    unsigned long ulLastBootUpgradeResetWibu;
    unsigned long ulPrevBootUpgradeReset;
    unsigned long ulPrevBootUpgradeResetWibu;
    unsigned int  uiBootUpgradeCRC;
    unsigned long ulLogErasure;                  // Log Erasure
    unsigned long ulLastLogErasure;
    unsigned long ulLastLogErasureReset;
    unsigned long ulLastLogErasureResetWibu;
    unsigned long ulPrevLogErasureReset;
    unsigned long ulPrevLogErasureResetWibu;
    unsigned int  uiLogErasureCRC;

} SystemCountersStruct;
```

3.4 Design constraints

3.4.1 Standards compliance

- All languages and operation systems must have an acceptable customer base (widespread use) and commercially available in English.

3.5 VAP1.28 Software system attributes

3.5.1 Reliability

- To ascertain that the software development is performed in a trustable technique, the development activities shall be compliant to the VSRS software engineering process CMM-01.

3.5.2 VAP1.28.1 Availability

- VAP1.28.1.1 DCM application shall remain in a safe and predictable condition while initializing and be fully functional less than 60 seconds after a power-up or reset and stay functional as long as it is powered.
- VAP1.28.1.3 To ensure availability, system must run under 1-second watchdog supervision; a failure to reset the watchdog timer within the pre-defined time shall trigger the re-initialization.

3.5.3 Security

- DCM Application shall be protected against malicious access, modification, destruction, or disclosure by having no user interface other than the PTA. See PTA SRS document #201111274 for its security implementation.

3.5.4 Maintainability

- Detailed documentation of the system shall be delivered to facilitate the maintenance of the software package.

3.5.5 Portability

- Not applicable.

APPENDIX A: CONFIGURABLE PARAMETERS

The DCM application accesses a configuration file particular to the project which contains all the values needed to describe the behavior of the DMS. All the parameters listed below will be defined in the Software Applications Configuration Specifications document, SACS. If a parameter is not be used in this project, its value will be set accordingly so that it is rendered inactive.

The configuration information is entirely defined in the Software Applications Configuration Specifications document, SACS.

Table 4-1: DCM Generic Parameters

MNEMONIC	Description	Range
SACS.Door open&close		
SACS.Door.DefaultOpeningDelay	After receiving an open command, the door will not move during the opening delay, and start to open only after the end of it. (Seconds)	1.0-2.0
SACS.Door.DefaultOpeningTime	The default time taken by the door to go from totally closed to within 3 inches of the fully open position. (Seconds)	SACS.Door.OpeningTime.Range
SACS.Door.OpeningTime.Range	The value of the opening time can be changed within the opening time range through the PTA. (Seconds)	N/A
SACS.Door.DefaultClosingDelay	After receiving a Close command, the door will not move during the Closing Delay, and start to close after the end of it. (Seconds)	1.0-2.5
SACS.Door.DefaultClosingTime	Default Time taken by the door to go from totally open to totally closed. (Seconds)	SACS.Door.ClosingTime.Range
SACS.Door.ClosingTime.Range	The closing time value can be changed within the closing time range. (Seconds)	N/A
SACS.Door.CrewSwitch		
SACS.Door.CrewSwitch.Outside.ClosingDelay	After receiving a Close command from the Outside Crew switch, the door will not move during the Crew Switch Outside Closing Delay, and start to close after the end of it. (Seconds)	0

MNEMONIC	Description	Range
SACS.Door.CrewSwitch.Inside.ClosingDelay	After receiving a Close command from the Inside Crew switch, the door will not move during the Crew Switch Outside Closing Delay, and start to close after the end of it. (Seconds)	0
SACS.Door.CrewSwitch.Outside.ClosingTime	The time taken by the door to go from totally open to totally close when commanded to close through the outside crew switch. (Seconds)	N/A
SACS.Door.CrewSwitch.Inside.ClosingTime	The time taken by the door to go from totally open to totally close when commanded to close through the inside crew switch. (Seconds)	N/A
SACS.Door.CrewSwitch.Outside.OpeningDelay	After receiving a Close command from the Outside Crew switch, the door will not move during the Crew Switch Outside Closing Delay, and start to close after the end of it. (Seconds)	0
SACS.Door.CrewSwitch.Inside.OpeningDelay	After receiving a Close command from the Inside Crew switch, the door will not move during the Crew Switch Outside Closing Delay, and start to close after the end of it. (Seconds)	0
SACS.Door.CrewSwitch.Outside.OpeningTime	The time taken by the door to go from totally close to totally open when commanded to close through the outside crew switch. (Seconds)	N/A
SACS.Door.CrewSwitch.Inside.OpeningTime	The time taken by the door to go from totally close to totally open when commanded to close through the inside crew switch. (Seconds)	N/A
SACS.Door.CrewSwitch.LinkTiming	When this parameter is equal to 1, the Crew Switches timing values are linked with the default door values for opening/closing delay & time. If the timing values are independent, the parameter is equal to 0.	0-1
SACS.Door.Obstruction		
SACS.Door.Obstruction.Distance	After detecting an obstruction, the door will move from the obstruction opening distance in the open direction.(Inches)	0-10
SACS.Door.Obstruction.Count	After detecting an obstacle, the door will try to open and re-close for a maximum number of time equals to the obstruction count	0-4

MNEMONIC	Description	Range
SACS.Door.Obstruction.EndBehavior	After having reached the maximum number of retries of closing the door, the door will adopt the behavior selected by the Obstruction End Behavior value. Value 1 means that the door will fully re-open.	0-2
SACS.Door.Obstruction.Range	The obstruction detection is working inside this obstruction range, measured in inches from the fully open position.(Inches)	0-10
SACS.Door.AudibleAlarmActivation	The behavior of the audible alarm upon opening or closure is selected by this parameter. Value 0 means the alarm will sounds as soon as the command is received, until the door stop moving	0-1
SACS.Door.VisualAlarmActivation	The behavior of the audible alarm upon opening or closure is selected by this parameter. Value 0 means the alarm will blink as soon as the close command is received, until the door stop moving	0-1
SACS.Door.SecondaryObstruction		
SACS.Door.SecondaryObstruction.Position	Position of the door, closed but not locked, stopped to detect a secondary obstruction. The position is measured in 10^{-2} inches from the locked position	TBD
SACS.Door.SecondaryObstruction.Activation	If equal to 1, the secondary obstruction function is activated, if equal to 0, desactivated.	Boolean
SACS.Door.SecondaryObstruction.Delay	Delay during which the door stay stopped in closed position to detect an obstruction.(Milliseconds)	TBD
SACS.Door.EmergencyHandle		
SACS.Door.EmergencyHandle.Close Power	If the parameter is equal to 1, Close power will be activated if motion trainline is active and the Emergency handle pulled. If the value is 0, close power will be inactive.	Boolean
SACS.Door.Snapshot		
SACS.Door.Snapshot.Period	The number of snapshot samples taken per second.	0 to 100
SACS.Door.Snapshot.Capacity	The maximum number of Snapshot samples which can be kept simultaneously in memory in a single Snapshot file.	0 to 41
SACS.Door.Snapshot.NbOfFiles	Maximum number of Snapshot Files saved in memory simultaneously.	0 to 20